

Reflection in .NET

The classes in the [System.Reflection](#) namespace, together with [System.Type](#), enable you to obtain information about loaded [assemblies](#) and the types defined within them, such as [classes](#), [interfaces](#), and value types (that is, [structures](#) and [enumerations](#)). You can also use reflection to create type instances at runtime, and to invoke and access them.

[Assemblies](#) contain modules, modules contain types, and types contain members. Reflection provides objects that encapsulate assemblies, modules, and types. You can use reflection to dynamically create an instance of a type, bind the type to an existing object, or get the type from an existing object. You can then invoke the type's methods or access its fields and properties. Typical uses of reflection include the following:

- Use [Assembly](#) to define and load assemblies, load modules that are listed in the assembly manifest, and locate a type from this assembly and create an instance of it.
- Use [Module](#) to discover information such as the assembly that contains the module and the classes in the module. You can also get all global methods or other specific, non-global methods defined on the module.
- Use [ConstructorInfo](#) to discover information such as the name, parameters, access modifiers (such as `public` or `private`), and implementation details (such as `abstract` or `virtual`) of a constructor. Use the [GetConstructors](#) or [GetConstructor](#) method of a [Type](#) to invoke a specific constructor.
- Use [MethodInfo](#) to discover information such as the name, return type, parameters, access modifiers, and implementation details (such as `abstract` or `virtual`) of a method. Use the [GetMethods](#) or [GetMethod](#) method of a [Type](#) to invoke a specific method.
- Use [FieldInfo](#) to discover information such as the name, access modifiers, and implementation details (such as `static`) of a field, and to get or set field values.
- Use [EventInfo](#) to discover information such as the name, event-handler data type, custom attributes, declaring type, and reflected type of an event, and to add or remove event handlers.
- Use [PropertyInfo](#) to discover information such as the name, data type, declaring type, reflected type, and read-only or writable status of a property, and to get or set property values.
- Use [ParameterInfo](#) to discover information such as a parameter's name, data type, whether a parameter is an input or output parameter, and the position of the parameter in a method signature.
- Use [CustomAttributeData](#) to discover information about custom attributes when you are working in the [MetadataLoadContext](#) or reflection-only context (.NET Framework). [CustomAttributeData](#) allows you to examine attributes without creating instances of them.

The classes of the [System.Reflection.Emit](#) namespace provide a specialized form of reflection that enables you to build types at runtime.

Reflection can also be used to create *type browsers*, which enable users to select types and then view the information about those types.

There are other uses for reflection. Compilers for languages such as JScript use reflection to construct symbol tables. The classes in the [System.Runtime.Serialization](#) namespace use reflection to access data and to determine which fields to persist. The classes in the [System.Runtime.Remoting](#) namespace use reflection indirectly through serialization.

Runtime types in reflection

Reflection provides classes, such as [Type](#) and [MethodInfo](#), to represent types, members, parameters, and other code entities. However, when you use reflection, you don't work directly with these classes, most of which are abstract (`MustInherit` in Visual Basic). Instead, you work with types provided by the common language runtime (CLR).

For example, when you use the C# `typeof` operator (`GetType` in Visual Basic) to obtain a [Type](#) object, the object is really a `RuntimeType`. `RuntimeType` derives from [Type](#) and provides implementations of all the abstract methods.

These runtime classes are `internal` (`Friend` in Visual Basic). They are not documented separately from their base classes, because their behavior is described by the base class documentation.

Reference

- [System.Type](#)
- [System.Reflection](#)
- [System.Reflection.Emit](#)

Security Considerations for Reflection

Article • 07/23/2022

Reflection provides the ability to obtain information about types and members, and to access members (that is, to call methods and constructors, to get and set property values, to add and remove event handlers, and so on). The use of reflection to obtain information about types and members is not restricted. All code can use reflection to perform the following tasks:

- Enumerate types and members, and examine their metadata.
- Enumerate and examine assemblies and modules.

Using reflection to access members, by contrast, is subject to restrictions. Beginning with the .NET Framework 4, only trusted code can use reflection to access security-critical members. Furthermore, only trusted code can use reflection to access nonpublic members that would not be directly accessible to compiled code. Finally, code that uses reflection to access a safe-critical member must have whatever permissions the safe-critical member demands, just as with compiled code.

Subject to necessary permissions, code can use reflection to perform the following kinds of access:

- Access public members that are not security-critical.
- Access nonpublic members that would be accessible to compiled code, if those members are not security-critical. Examples of such nonpublic members include:
 - Protected members of the calling code's base classes. (In reflection, this is referred to as family-level access.)
 - `internal` members (`Friend` members in Visual Basic) in the calling code's assembly. (In reflection, this is referred to as assembly-level access.)
 - Private members of other instances of the class that contains the calling code.

For example, code that is run in a sandboxed application domain is limited to the access described in this list, unless the application domain grants additional permissions.

Starting with the .NET Framework 2.0 Service Pack 1, attempting to access members that are normally inaccessible generates a demand for the grant set of the target object plus [ReflectionPermission](#) with the [ReflectionPermissionFlag.MemberAccess](#) flag. Code that is running with full trust (for example, code in an application that is launched from the

command line) can always satisfy these permissions. (This is subject to limitations in accessing security-critical members, as described later in this article.)

Optionally, a sandboxed application domain can grant [ReflectionPermission](#) with the [ReflectionPermissionFlag.MemberAccess](#) flag, as described in the section [Accessing Members That Are Normally Inaccessible](#), later in this article.

Accessing Security-Critical Members

A member is security-critical if it has the [SecurityCriticalAttribute](#), if it belongs to a type that has the [SecurityCriticalAttribute](#), or if it is in a security-critical assembly. Beginning with the .NET Framework 4, the rules for accessing security-critical members are as follows:

- Transparent code cannot use reflection to access security-critical members, even if the code is fully trusted. A [MethodAccessException](#), [FieldAccessException](#), or [TypeAccessException](#) is thrown.
- Code that is running with partial trust is treated as transparent.

These rules are the same whether a security-critical member is accessed directly by compiled code, or accessed by using reflection.

Application code that is run from the command line runs with full trust. As long as it is not marked as transparent, it can use reflection to access security-critical members. When the same code is run with partial trust (for example, in a sandboxed application domain) the assembly's trust level determines whether it can access security-critical code: If the assembly has a strong name and is installed in the global assembly cache, it is a trusted assembly and can call security-critical members. If it is not trusted, it becomes transparent even though it was not marked as transparent, and it cannot access security-critical members.

Reflection and Transparency

Beginning with the .NET Framework 4, the common language runtime determines the transparency level of a type or member from several factors, including the trust level of the assembly and the trust level of the application domain. Reflection provides the [IsSecurityCritical](#), [IsSecuritySafeCritical](#), and [IsSecurityTransparent](#) properties to enable you to discover the transparency level of a type. The following table shows the valid combinations of these properties.

Security level	IsSecurityCritical	IsSecuritySafeCritical	IsSecurityTransparent
Critical	true	false	false
Safe-critical	true	true	false
Transparent	false	false	true

Using these properties is much simpler than examining the security annotations of an assembly and its types, checking the current trust level, and attempting to duplicate the runtime's rules. For example, the same type can be security-critical when it is run from the command line, or security-transparent when it is run in a sandboxed application domain.

There are similar properties on the [MethodBase](#), [FieldInfo](#), [TypeBuilder](#), [MethodBuilder](#), and [DynamicMethod](#) classes. (For other reflection and reflection emit abstractions, security attributes are applied to the associated methods; for example, in the case of properties they are applied to the property accessors.)

Accessing Members That Are Normally Inaccessible

To use reflection to invoke members that are inaccessible according to the accessibility rules of the common language runtime, your code must be granted one of two permissions:

- To allow code to invoke any nonpublic member: Your code must be granted [ReflectionPermission](#) with the [ReflectionPermissionFlag.MemberAccess](#) flag.

ⓘ Note

By default, security policy denies this permission to code that originates from the Internet. This permission should never be granted to code that originates from the Internet.

- To allow code to invoke any nonpublic member, as long as the grant set of the assembly that contains the invoked member is the same as, or a subset of, the grant set of the assembly that contains the invoking code: Your code must be granted [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag.

For example, suppose you grant an application domain Internet permissions plus [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag, and then run an Internet application with two assemblies, A and B.

- Assembly A can use reflection to access private members of assembly B, because the grant set of assembly B does not include any permissions that A has not been granted.
- Assembly A cannot use reflection to access private members of .NET Framework assemblies such as mscorlib.dll, because mscorlib.dll is fully trusted and therefore has permissions that have not been granted to assembly A. A [MemberAccessException](#) is thrown when code access security walks the stack at run time.

Serialization

For serialization, [SecurityPermission](#) with the [SecurityPermissionAttribute.SerializationFormatter](#) flag provides the ability to get and set members of serializable types, regardless of accessibility. This permission enables code to discover and change the private state of an instance. (In addition to being granted the appropriate permissions, the type must be [marked](#) as serializable in metadata.)

Parameters of Type MethodInfo

Avoid writing public members that take [MethodInfo](#) parameters, especially for trusted code. Such members might be more vulnerable to malicious code. For example, consider a public member in highly trusted code that takes a [MethodInfo](#) parameter. Assume that the public member indirectly calls the [Invoke](#) method on the supplied parameter. If the public member does not perform the necessary permission checks, the call to the [Invoke](#) method will always succeed, because the security system determines that the caller is highly trusted. Even if malicious code does not have the permission to directly invoke the method, it can still do so indirectly by calling the public member.

Version Information

- Beginning with the .NET Framework 4, transparent code cannot use reflection to access security-critical members.
- The [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag is introduced in the .NET Framework 2.0 Service Pack 1. Earlier versions of the .NET Framework require

the [ReflectionPermissionFlag.MemberAccess](#) flag for code that uses reflection to access nonpublic members. This is a permission that should never be granted to partially trusted code.

- Beginning with .NET Framework 2.0, using reflection to obtain information about nonpublic types and members does not require any permissions. In earlier versions, [ReflectionPermission](#) with the [ReflectionPermissionFlag.TypeInformation](#) flag is required.

See also

- [ReflectionPermissionFlag](#)
- [ReflectionPermission](#)
- [SecurityPermission](#)
- [Code Access Security](#)
- [Security Issues in Reflection Emit](#)
- [Viewing Type Information](#)
- [Applying Attributes](#)
- [Accessing Custom Attributes](#)

How to: Load assemblies into the reflection-only context

Article • 03/30/2024

The reflection-only load context allows you to examine assemblies compiled for other platforms or for other versions of .NET. Code loaded into this context can only be examined; it can't be executed. This means that objects can't be created, because constructors can't be executed. Because the code can't be executed, dependencies are not automatically loaded. If you need to examine them, you must load them yourself.

To load an assembly into the reflection-only load context

1. Use the [ReflectionOnlyLoad\(String\)](#) method overload to load the assembly given its display name, or the [ReflectionOnlyLoadFrom](#) method to load the assembly given its path. If the assembly is a binary image, use the [ReflectionOnlyLoad\(Byte\[\]\)](#) method overload.

ⓘ Note

You can't use the reflection-only context to load a version of mscorlib.dll from a version of the .NET Framework other than the version in the execution context.

2. If the assembly has dependencies, the [ReflectionOnlyLoad](#) method does not load them. If you need to examine them, you must load them yourself.
3. Determine whether an assembly is loaded into the reflection-only context by using the assembly's [ReflectionOnly](#) property.
4. If attributes have been applied to the assembly or to types in the assembly, examine those attributes by using the [CustomAttributeData](#) class to ensure that no attempt is made to execute code in the reflection-only context. Use the appropriate overload of the [CustomAttributeData.GetCustomAttributes](#) method to obtain [CustomAttributeData](#) objects representing the attributes applied to an assembly, member, module, or parameter.

ⓘ Note

Attributes applied to the assembly or to its contents might be defined in the assembly, or they might be defined in another assembly loaded into the reflection-only context. There is no way to tell in advance where the attributes are defined.

Example

The following code example shows how to examine the attributes applied to an assembly loaded into the reflection-only context.

The code example defines a custom attribute with two constructors and one property. The attribute is applied to the assembly, to a type declared in the assembly, to a method of the type, and to a parameter of the method. When executed, the assembly loads itself into the reflection-only context and displays information about the custom attributes that were applied to it and to the types and members it contains.

❗ Note

To simplify the code example, the assembly loads and examines itself. Normally, you would not expect to find the same assembly loaded into both the execution context and the reflection-only context.

C#

```
using System;
using System.Reflection;
using System.Collections.Generic;
using System.Collections.ObjectModel;

// The example attribute is applied to the assembly.
[assembly:Example(ExampleKind.ThirdKind, Note="This is a note on the
assembly.")]

// An enumeration used by the ExampleAttribute class.
public enum ExampleKind
{
    FirstKind,
    SecondKind,
    ThirdKind,
    FourthKind
};

// An example attribute. The attribute can be applied to all
// targets, from assemblies to parameters.
//
```

```

[AttributeUsage(AttributeTargets.All)]
public class ExampleAttribute : Attribute
{
    // Data for properties.
    private ExampleKind kindValue;
    private string noteValue;
    private string[] arrayStrings;
    private int[] arrayNumbers;

    // Constructors. The parameterless constructor (.ctor) calls
    // the constructor that specifies ExampleKind and an array of
    // strings, and supplies the default values.
    //
    public ExampleAttribute(ExampleKind initKind, string[] initStrings)
    {
        kindValue = initKind;
        arrayStrings = initStrings;
    }
    public ExampleAttribute(ExampleKind initKind) : this(initKind, null) {}
    public ExampleAttribute() : this(ExampleKind.FirstKind, null) {}

    // Properties. The Note and Numbers properties must be read/write, so
they
    // can be used as named parameters.
    //
    public ExampleKind Kind { get { return kindValue; }}
    public string[] Strings { get { return arrayStrings; }}
    public string Note
    {
        get { return noteValue; }
        set { noteValue = value; }
    }
    public int[] Numbers
    {
        get { return arrayNumbers; }
        set { arrayNumbers = value; }
    }
}

// The example attribute is applied to the test class.
//
[Example(ExampleKind.SecondKind,
    new string[] { "String array argument, line 1",
        "String array argument, line 2",
        "String array argument, line 3" },
    Note="This is a note on the class.",
    Numbers = new int[] { 53, 57, 59 })]
public class Test
{
    // The example attribute is applied to a method, using the
    // parameterless constructor and supplying a named argument.
    // The attribute is also applied to the method parameter.
    //
    [Example(Note="This is a note on a method.")]
    public void TestMethod([Example] object arg) { }
}

```

```

// Main() gets objects representing the assembly, the test
// type, the test method, and the method parameter. Custom
// attribute data is displayed for each of these.
//
public static void Main()
{
    Assembly asm = Assembly.ReflectionOnlyLoad("Source");
    Type t = asm.GetType("Test");
    MethodInfo m = t.GetMethod("TestMethod");
    ParameterInfo[] p = m.GetParameters();

    Console.WriteLine($"\\r\\nAttributes for assembly: '{asm}'");
    ShowAttributeData(CustomAttributeData.GetCustomAttributes(asm));
    Console.WriteLine($"\\r\\nAttributes for type: '{t}'");
    ShowAttributeData(CustomAttributeData.GetCustomAttributes(t));
    Console.WriteLine($"\\r\\nAttributes for member: '{m}'");
    ShowAttributeData(CustomAttributeData.GetCustomAttributes(m));
    Console.WriteLine($"\\r\\nAttributes for parameter: '{p}'");
    ShowAttributeData(CustomAttributeData.GetCustomAttributes(p[0]));
}

private static void ShowAttributeData(
    IList<CustomAttributeData> attributes)
{
    foreach( CustomAttributeData cad in attributes )
    {
        Console.WriteLine($"    {cad}");
        Console.WriteLine($"        Constructor: '{cad.Constructor}'");

        Console.WriteLine("        Constructor arguments:");
        foreach( CustomAttributeTypedArgument cata
            in cad.ConstructorArguments )
        {
            ShowValueOrArray(cata);
        }

        Console.WriteLine("        Named arguments:");
        foreach( CustomAttributeNamedArgument cana
            in cad.NamedArguments )
        {
            Console.WriteLine($"            MemberInfo:
'{cana.MemberInfo}'");
            ShowValueOrArray(cana.TypedValue);
        }
    }
}

private static void ShowValueOrArray(CustomAttributeTypedArgument cata)
{
    if (cata.Value.GetType() ==
typeof(ReadOnlyCollection<CustomAttributeTypedArgument>))
    {
        Console.WriteLine($"        Array of '{cata.ArgumentType}':");
    }
}

```

```

        foreach (CustomAttributeTypedArgument cataElement in
            (ReadOnlyCollection<CustomAttributeTypedArgument>)
cata.Value)
        {
            Console.WriteLine($"                Type:
'{cataElement.ArgumentType}' Value: '{cataElement.Value}");
        }
    }
    else
    {
        Console.WriteLine($"                Type: '{cata.ArgumentType}' Value:
'{cata.Value}");
    }
}
}

```

/* This code example produces output similar to the following:

Attributes for assembly: 'source, Version=0.0.0.0, Culture=neutral,
PublicKeyToken=null'

[System.Runtime.CompilerServices.CompilationRelaxationsAttribute((Int32)8)]

Constructor: 'Void .ctor(Int32)'

Constructor arguments:

Type: 'System.Int32' Value: '8'

Named arguments:

[System.Runtime.CompilerServices.RuntimeCompatibilityAttribute(WrapNonExcept
ionThrows = True)]

Constructor: 'Void .ctor()'

Constructor arguments:

Named arguments:

MemberInfo: 'Boolean WrapNonExceptionThrows'

Type: 'System.Boolean' Value: 'True'

[ExampleAttribute((ExampleKind)2, Note = "This is a note on the
assembly.")]

Constructor: 'Void .ctor(ExampleKind)'

Constructor arguments:

Type: 'ExampleKind' Value: '2'

Named arguments:

MemberInfo: 'System.String Note'

Type: 'System.String' Value: 'This is a note on the assembly.'

Attributes for type: 'Test'

[ExampleAttribute((ExampleKind)1, new String[3] { "String array argument,
line 1", "String array argument, line 2", "String array argument, line 3" },
Note = "This is a note on the class.", Numbers = new Int32[3] { 53, 57, 59
})]

Constructor: 'Void .ctor(ExampleKind, System.String[])'

Constructor arguments:

Type: 'ExampleKind' Value: '1'

Array of 'System.String':

Type: 'System.String' Value: 'String array argument, line 1'

Type: 'System.String' Value: 'String array argument, line 2'

Type: 'System.String' Value: 'String array argument, line 3'

```

    Named arguments:
      MemberInfo: 'System.String Note'
      Type: 'System.String' Value: 'This is a note on the class.'
      MemberInfo: 'Int32[] Numbers'
      Array of 'System.Int32[]':
        Type: 'System.Int32' Value: '53'
        Type: 'System.Int32' Value: '57'
        Type: 'System.Int32' Value: '59'

    Attributes for member: 'Void TestMethod(System.Object)'
    [ExampleAttribute(Note = "This is a note on a method.")]
    Constructor: 'Void .ctor()'
    Constructor arguments:
    Named arguments:
      MemberInfo: 'System.String Note'
      Type: 'System.String' Value: 'This is a note on a method.'

    Attributes for parameter: 'System.Object arg'
    [ExampleAttribute()]
    Constructor: 'Void .ctor()'
    Constructor arguments:
    Named arguments:
*/

```

See also

- [ReflectionOnlyLoad](#)
- [ReflectionOnly](#)
- [CustomAttributeData](#)

Reflection in the .NET Framework for Windows Store Apps

Article • 07/23/2022

Starting with .NET Framework 4.5, the .NET Framework includes a set of reflection types and members for use in Windows 8.x Store apps. These types and members are available in the full .NET Framework as well as in the .NET for Windows Store apps. This document explains the major differences between these and their counterparts in the .NET Framework 4 and earlier versions.

If you are creating a Windows 8.x Store app, you must use the reflection types and members in the .NET for Windows 8.x Store apps. These types and members are also available, but not required, for use in desktop apps, so you can use the same code for both types of apps.

TypeInfo and Assembly Loading

In the .NET for Windows 8.x Store apps, the [TypeInfo](#) class contains some of the functionality of the .NET Framework 4 [Type](#) class. A [Type](#) object represents a reference to a type definition, whereas a [TypeInfo](#) object represents the type definition itself. This enables you to manipulate [Type](#) objects without necessarily requiring the runtime to load the assembly they reference. Getting the associated [TypeInfo](#) object forces the assembly to load.

[TypeInfo](#) contains many of the members available on [Type](#), and many of the reflection properties in the .NET for Windows 8.x Store apps return collections of [TypeInfo](#) objects. To get a [TypeInfo](#) object from a [Type](#) object, use the [GetTypeInfo](#) method.

Query Methods

In the .NET for Windows 8.x Store apps, you use the reflection properties that return [IEnumerable<T>](#) collections instead of methods that return arrays. Reflection contexts can implement lazy traversal of these collections for large assemblies or types.

The reflection properties return only the declared methods on a particular object instead of traversing the inheritance tree. Moreover, they do not use [BindingFlags](#) parameters for filtering. Instead, filtering takes place in user code, by using LINQ queries on the returned collections. For reflection objects that originate with the runtime (for example, as the result of `typeof(Object)`), traversing the inheritance tree is best accomplished by

using the helper methods of the [RuntimeReflectionExtensions](#) class. Consumers of objects from customized reflection contexts cannot use these methods, and must traverse the inheritance tree themselves.

Restrictions

In a Windows 8.x Store app, access to some .NET Framework types and members is restricted. For example, you cannot call .NET Framework methods that are not included in .NET for Windows 8.x Store apps, by using a [MethodInfo](#) object. In addition, certain types and members that are not considered safe within the context of a Windows 8.x Store app are blocked, as are [Marshal](#) and [WindowsRuntimeMarshal](#) members. This restriction affects only .NET Framework types and members; you can call your code or third-party code as you normally would.

Example

This example uses the reflection types and members in the .NET for Windows 8.x Store apps to retrieve the methods and properties of the [Calendar](#) type, including inherited methods and properties. To run this code, paste it into the code file for a Windows 8.x Store page that contains a [Windows.UI.Xaml.Controls.TextBlock](#) control named `textblock1` in a project named Reflection. If you paste this code inside a project with a different name, just make sure you change the namespace name to match your project.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Navigation;
using System.Reflection;
using System.Globalization;
using System.Text;

namespace Reflection
{
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
        }
    }
}
```

```

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    TypeInfo t = typeof(Calendar).GetTypeInfo();
    IEnumerable<PropertyInfo> pList = t.DeclaredProperties;
    IEnumerable<MethodInfo> mList = t.DeclaredMethods;

    StringBuilder sb = new StringBuilder();

    sb.Append("Properties:");
    foreach (PropertyInfo p in pList)
    {
        sb.Append("\n" + p.DeclaringType.Name + ": " + p.Name);
    }
    sb.Append("\nMethods:");
    foreach (MethodInfo m in mList)
    {
        sb.Append("\n" + m.DeclaringType.Name + ": " + m.Name);
    }

    textblock1.Text = sb.ToString();
}
}
}

```

See also

- [Reflection](#)

Emit dynamic methods and assemblies

This section describes a set of managed types in the [System.Reflection.Emit](#) namespace that allow a compiler or tool to emit metadata and common intermediate language (CIL) at runtime and optionally generate a portable executable (PE) file on disk. Script engines and compilers are the primary users of this namespace. In this section, the functionality provided by the [System.Reflection.Emit](#) namespace is referred to as *reflection emit*.

Reflection emit provides the following capabilities:

- Define lightweight global methods at runtime, using the [DynamicMethod](#) class, and execute them using delegates.
- Define assemblies at runtime and then run them and/or save them to disk.
- Define assemblies at runtime, run them, and then unload them and allow garbage collection to reclaim their resources.
- Define modules in new assemblies at runtime and then run and/or save them to disk.
- Define types in modules at runtime, create instances of these types, and invoke their methods.
- Define symbolic information for defined modules that can be used by tools such as debuggers and code profilers.

In addition to the managed types in the [System.Reflection.Emit](#) namespace, there are unmanaged metadata interfaces that are described in [Metadata interfaces \(.NET Framework\)](#) and [Metadata interfaces \(.NET\)](#). Managed reflection emit provides stronger semantic error checking and a higher level of abstraction of the metadata than the unmanaged metadata interfaces.

Another useful resource for working with metadata and CIL is the Common Language Infrastructure (CLI) documentation, especially "Partition II: Metadata Definition and Semantics" and "Partition III: CIL Instruction Set". The documentation is available online at the [Ecma Web site](#) [↗](#).

Reference

[OpCodes](#)

Catalogs the CIL instruction codes you can use to build method bodies.

[System.Reflection.Emit](#)

Contains managed classes used to emit dynamic methods, assemblies, and types.

[Type](#)

Describes the [Type](#) class, which represents types in managed reflection and reflection emit, and

which is key to the use of these technologies.

[System.Reflection](#)

Contains managed classes used to explore metadata and managed code.

Last updated on 12/03/2025

Security Issues in Reflection Emit

Article • 09/15/2021

.NET Framework provides three ways to emit common intermediate language (CIL), each with its own security issues:

- [Dynamic Assemblies](#)
- [Anonymously Hosted Dynamic Methods](#)
- [Dynamic Methods Associated with Existing Assemblies](#)

Regardless of the way you generate dynamic code, executing the generated code requires all the permissions that are required by the types and methods the generated code uses.

ⓘ Note

The permissions that are required for reflecting on code and emitting code have changed with succeeding releases of .NET Framework. See [Version Information](#), later in this article.

Dynamic Assemblies

Dynamic assemblies are created by using overloads of the [AppDomain.DefineDynamicAssembly](#) method. Most overloads of this method are deprecated in .NET Framework 4, because of the elimination of machine-wide security policy. The remaining overloads can be executed by any code, regardless of trust level. These overloads fall into two groups: those that specify a list of attributes to apply to the dynamic assembly when it is created, and those that do not. If you do not specify the transparency model for the assembly, by applying the [SecurityRulesAttribute](#) attribute when you create it, the transparency model is inherited from the emitting assembly.

ⓘ Note

Attributes that you apply to the dynamic assembly after it is created, by using the [SetCustomAttribute](#) method, do not take effect until the assembly has been saved to disk and loaded into memory again.

Code in a dynamic assembly can access visible types and members in other assemblies.

ⓘ Note

Dynamic assemblies do not use the [ReflectionPermissionFlag.MemberAccess](#) and [ReflectionPermissionFlag.RestrictedMemberAccess](#) flags that allow dynamic methods to access nonpublic types and members.

Transient dynamic assemblies are created in memory and never saved to disk, so they require no file access permissions. Saving a dynamic assembly to disk requires [FileOPermission](#) with the appropriate flags.

Generating Dynamic Assemblies from Partially Trusted Code

Consider the conditions in which an assembly with Internet permissions can generate a transient dynamic assembly and execute its code:

- The dynamic assembly uses only public types and members of other assemblies.
- The permissions demanded by those types and members are included in the grant set of the partially trusted assembly.
- The assembly is not saved to disk.
- Debug symbols are not generated. (`Internet` and `LocalIntranet` permission sets do not include the necessary permissions.)

Anonymously Hosted Dynamic Methods

Anonymously hosted dynamic methods are created by using the two [DynamicMethod](#) constructors that do not specify an associated type or module, [DynamicMethod\(String, Type, Type\[\]\)](#) and [DynamicMethod\(String, Type, Type\[\], Boolean\)](#). These constructors place the dynamic methods in a system-provided, fully trusted, security-transparent assembly. No permissions are required to use these constructors or to emit code for the dynamic methods.

Instead, when an anonymously hosted dynamic method is created, the call stack is captured. When the method is constructed, security demands are made against the captured call stack.

ⓘ Note

Conceptually, demands are made during the construction of the method. That is, demands could be made as each CIL instruction is emitted. In the current implementation, all demands are made when the [DynamicMethod.CreateDelegate](#) method is called or when the just-in-time (JIT) compiler is invoked, if the method is invoked without calling [CreateDelegate](#).

If the application domain permits it, anonymously hosted dynamic methods can skip JIT visibility checks, subject to the following restriction: The nonpublic types and members accessed by an anonymously hosted dynamic method must be in assemblies whose grant sets are equal to, or subsets of, the grant set of the emitting call stack. This restricted ability to skip JIT visibility checks is enabled if the application domain grants [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag.

- If your method uses only public types and members, no permissions are required during construction.
- If you specify that JIT visibility checks should be skipped, the demand that is made when the method is constructed includes [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag and the grant set of the assembly that contains the nonpublic member that is being accessed.

Because the grant set of the nonpublic member is taken into consideration, partially trusted code that has been granted [ReflectionPermissionFlag.RestrictedMemberAccess](#) cannot elevate its privileges by executing nonpublic members of trusted assemblies.

As with any other emitted code, executing the dynamic method requires whatever permissions are demanded by the methods the dynamic method uses.

The system assembly that hosts anonymously-hosted dynamic methods uses the [SecurityRuleSet.Level1](#) transparency model, which is the transparency model that was used in .NET Framework before .NET Framework 4.

For more information, see the [DynamicMethod](#) class.

Generating Anonymously Hosted Dynamic Methods from Partially Trusted Code

Consider the conditions in which an assembly with Internet permissions can generate an anonymously hosted dynamic method and execute it:

- The dynamic method uses only public types and members. If its grant set includes [ReflectionPermissionFlag.RestrictedMemberAccess](#), it can use nonpublic types and

members of any assembly whose grant set is equal to, or a subset of, the grant set of the emitting assembly.

- The permissions that are required by all the types and members used by the dynamic method are included in the grant set of the partially trusted assembly.

 Note

Dynamic methods do not support debug symbols.

Dynamic Methods Associated with Existing Assemblies

To associate a dynamic method with a type or module in an existing assembly, use any of the [DynamicMethod](#) constructors that specify the associated type or module. The permissions that are required to call these constructors vary, because associating a dynamic method with an existing type or module gives the dynamic method access to nonpublic types and members:

- A dynamic method that is associated with a type has access to all members of that type, even private members, and to all internal types and members in the assembly that contains the associated type.
- A dynamic method that is associated with a module has access to all the `internal` types and members (`Friend` in Visual Basic, `assembly` in common language runtime metadata) in the module.

In addition, you can use a constructor that specifies the ability to skip the visibility checks of the JIT compiler. Doing so gives your dynamic method access to all types and members in all assemblies, regardless of access level.

The permissions demanded by the constructor depend on how much access you decide to give your dynamic method:

- If your method uses only public types and members, and you associate it with your own type or your own module, no permissions are required.
- If you specify that JIT visibility checks should be skipped, the constructor demands [ReflectionPermission](#) with the [ReflectionPermissionFlag.MemberAccess](#) flag.
- If you associate the dynamic method with another type, even another type in your own assembly, the constructor demands [ReflectionPermission](#) with the

[ReflectionPermissionFlag.MemberAccess](#) flag and [SecurityPermission](#) with the [SecurityPermissionFlag.ControlEvidence](#) flag.

- If you associate the dynamic method with a type or module in another assembly, the constructor demands two things: [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag, and the grant set of the assembly that contains the other module. That is, your call stack must include all the permissions in the grant set of the target module, plus [ReflectionPermissionFlag.RestrictedMemberAccess](#).

ⓘ Note

For backward compatibility, if the demand for the target grant set plus [ReflectionPermissionFlag.RestrictedMemberAccess](#) fails, the constructor demands [SecurityPermission](#) with the [SecurityPermissionFlag.ControlEvidence](#) flag.

Although the items in this list are described in terms of the grant set of the emitting assembly, remember that the demands are made against the full call stack, including the application domain boundary.

For more information, see the [DynamicMethod](#) class.

Generating Dynamic Methods from Partially Trusted Code

ⓘ Note

The recommended way to generate dynamic methods from partially trusted code is to use [Anonymously Hosted Dynamic Methods](#).

Consider the conditions in which an assembly with Internet permissions can generate a dynamic method and execute it:

- Either the dynamic method is associated with the module or type that emits it, or its grant set includes [ReflectionPermissionFlag.RestrictedMemberAccess](#) and it is associated with a module in an assembly whose grant set is equal to, or a subset of, the grant set of the emitting assembly.
- The dynamic method uses only public types and members. If its grant set includes [ReflectionPermissionFlag.RestrictedMemberAccess](#) and it is associated with a module in an assembly whose grant set is equal to, or a subset of, the grant set of

the emitting assembly, it can use types and members marked `internal` (`Friend` in Visual Basic, `assembly` in common language runtime metadata) in the associated module.

- The permissions demanded by all the types and members used by the dynamic method are included in the grant set of the partially trusted assembly.
- The dynamic method does not skip JIT visibility checks.

ⓘ Note

Dynamic methods do not support debug symbols.

Version Information

Starting with .NET Framework 4, machine-wide security policy is eliminated and security transparency becomes the default enforcement mechanism.

Starting with .NET Framework 2.0 Service Pack 1, [ReflectionPermission](#) with the [ReflectionPermissionFlag.ReflectionEmit](#) flag is no longer required when emitting dynamic assemblies and dynamic methods. This flag is required in all earlier versions of .NET Framework.

ⓘ Note

[ReflectionPermission](#) with the [ReflectionPermissionFlag.ReflectionEmit](#) flag is included by default in the `FullTrust` and `LocalIntranet` named permission sets, but not in the `Internet` permission set. Therefore, in earlier versions of .NET Framework, a library can be used with Internet permissions only if it executes an [Assert](#) for [ReflectionEmit](#). Such libraries require careful security review because coding errors could result in security holes. .NET Framework 2.0 SP1 allows code to be emitted in partial trust scenarios without issuing any security demands, because generating code is not inherently a privileged operation. That is, the generated code has no more permissions than the assembly that emits it. This allows libraries that emit code to be security transparent and removes the need to assert [ReflectionEmit](#), which simplifies the task of writing a secure library.

In addition, .NET Framework 2.0 SP1 introduces the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag for accessing nonpublic types and members from partially trusted dynamic methods. Earlier versions of the .NET

Framework require the [ReflectionPermissionFlag.MemberAccess](#) flag for dynamic methods that access nonpublic types and members; this is a permission that should never be granted to partially trusted code.

Finally, .NET Framework 2.0 SP1 introduces anonymously hosted methods.

Obtaining Information on Types and Members

Starting with .NET Framework 2.0, no permissions are required to obtain information about nonpublic types and members. Reflection is used to obtain information needed to emit dynamic methods. For example, [MethodInfo](#) objects are used to emit method calls. Earlier versions of .NET Framework require [ReflectionPermission](#) with the [ReflectionPermissionFlag.TypeInformation](#) flag. For more information, see [Security Considerations for Reflection](#).

See also

- [Security Considerations for Reflection](#)
- [Emitting Dynamic Methods and Assemblies](#)

Walkthrough: Emitting Code in Partial Trust Scenarios

Article • 03/11/2022

Reflection emit uses the same API set in full or partial trust, but some features require special permissions in partially trusted code. In addition, reflection emit has a feature, anonymously hosted dynamic methods, that is designed to be used with partial trust and by security-transparent assemblies.

ⓘ Note

Before .NET Framework 3.5, emitting code required [ReflectionPermission](#) with the [ReflectionPermissionFlag.ReflectionEmit](#) flag. This permission is included by default in the `FullTrust` and `Intranet` named permission sets, but not in the `Internet` permission set. Therefore, a library could be used from partial trust only if it had the [SecurityCriticalAttribute](#) attribute and also executed an [Assert](#) method for [ReflectionEmit](#). Such libraries require careful security review because coding errors could result in security holes. The .NET Framework 3.5 allows code to be emitted in partial trust scenarios without issuing any security demands, because generating code is not inherently a privileged operation. That is, the generated code has no more permissions than the assembly that emits it. This enables libraries that emit code to be security-transparent and removes the need to assert [ReflectionEmit](#), so that writing a secure library does not require such a thorough security review.

This walkthrough illustrates the following tasks:

- [Setting up a simple sandbox for testing partially trusted code.](#)

ⓘ Important

This is a simple way to experiment with code in partial trust. To run code that actually comes from untrusted locations, see [How to: Run Partially Trusted Code in a Sandbox](#).

- [Running code in partially trusted application domains.](#)
- [Using anonymously hosted dynamic methods to emit and execute code in partial trust.](#)

For more information about emitting code in partial trust scenarios, see [Security Issues in Reflection Emit](#).

For a complete listing of the code shown in these procedures, see the [Example section](#) at the end of this walkthrough.

Setting up Partially Trusted Locations

The following two procedures show how to set up locations from which you can test code with partial trust.

- The first procedure shows how to create a sandboxed application domain in which code is granted Internet permissions.
- The second procedure shows how to add [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag to a partially trusted application domain, to enable access to private data in assemblies of equal or lesser trust.

Creating Sandboxed Application Domains

To create an application domain in which your assemblies run with partial trust, you must specify the set of permissions to be granted to the assemblies by using the [AppDomain.CreateDomain\(String, Evidence, AppDomainSetup, PermissionSet, StrongName\[\]\)](#) method overload to create the application domain. The easiest way to specify the grant set is to retrieve a named permission set from security policy.

The following procedure creates a sandboxed application domain that runs your code with partial trust, to test scenarios in which emitted code can access only public members of public types. A subsequent procedure shows how to add [RestrictedMemberAccess](#), to test scenarios in which emitted code can access nonpublic types and members in assemblies that are granted equal or lesser permissions.

To create an application domain with partial trust

1. Create a permission set to grant to the assemblies in the sandboxed application domain. In this case, the permission set of the Internet zone is used.

C#

```
Evidence ev = new Evidence();  
ev.AddHostEvidence(new Zone(SecurityZone.Internet));
```

```
PermissionSet pset = new NamedPermissionSet("Internet",  
SecurityManager.GetStandardSandbox(ev));
```

2. Create an `AppDomainSetup` object to initialize the application domain with an application path.

Important

For simplicity, this code example uses the current folder. To run code that actually comes from the Internet, use a separate folder for the untrusted code, as described in [How to: Run Partially Trusted Code in a Sandbox](#).

C#

```
AppDomainSetup adSetup = new AppDomainSetup();  
adSetup.ApplicationBase = ".";
```

3. Create the application domain, specifying the application domain setup information and the grant set for all assemblies that execute in the application domain.

C#

```
AppDomain ad = AppDomain.CreateDomain("Sandbox", ev, adSetup, pset,  
null);
```

The last parameter of the `AppDomain.CreateDomain(String, Evidence, AppDomainSetup, PermissionSet, StrongName[])` method overload enables you to specify a set of assemblies that are to be granted full trust, instead of the grant set of the application domain. You do not have to specify the .NET Framework assemblies that your application uses, because those assemblies are in the global assembly cache. Assemblies in the global assembly cache are always fully trusted. You can use this parameter to specify strong-named assemblies that are not in the global assembly cache.

Adding RestrictedMemberAccess to Sandboxed Domains

Host applications can allow anonymously hosted dynamic methods to have access to private data in assemblies that have trust levels equal to or less than the trust level of the assembly that emits the code. To enable this restricted ability to skip just-in-time

(JIT) visibility checks, the host application adds a [ReflectionPermission](#) object with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) (RMA) flag to the grant set.

For example, a host might grant Internet applications Internet permissions plus RMA, so that an Internet application can emit code that accesses private data in its own assemblies. Because the access is limited to assemblies of equal or lesser trust, an Internet application cannot access members of fully trusted assemblies such as .NET Framework assemblies.

ⓘ Note

To prevent elevation of privilege, stack information for the emitting assembly is included when anonymously hosted dynamic methods are constructed. When the method is invoked, the stack information is checked. Thus, an anonymously hosted dynamic method that is invoked from fully trusted code is still limited to the trust level of the emitting assembly.

To create an application domain with partial trust plus RMA

1. Create a new [ReflectionPermission](#) object with the [RestrictedMemberAccess](#) (RMA) flag, and use the [PermissionSet.SetPermission](#) method to add the permission to the grant set.

C#

```
pset.SetPermission(  
    new ReflectionPermission(  
        ReflectionPermissionFlag.RestrictedMemberAccess));
```

The [AddPermission](#) method adds the permission to the grant set if it is not already included. If the permission is already included in the grant set, the specified flags are added to the existing permission.

ⓘ Note

RMA is a feature of anonymously hosted dynamic methods. When ordinary dynamic methods skip JIT visibility checks, the emitted code requires full trust.

2. Create the application domain, specifying the application domain setup information and the grant set.

C#

```
ad = AppDomain.CreateDomain("Sandbox2", ev, adSetup, pset, null);
```

Running Code in Sandboxed Application Domains

The following procedure explains how to define a class by using methods that can be executed in an application domain, how to create an instance of the class in the domain, and how to execute its methods.

To define and execute a method in an application domain

1. Define a class that derives from [MarshalByRefObject](#). This enables you to create instances of the class in other application domains and to make method calls across application domain boundaries. The class in this example is named `Worker`.

C#

```
public class Worker : MarshalByRefObject  
{
```

2. Define a public method that contains the code you want to execute. In this example, the code emits a simple dynamic method, creates a delegate to execute the method, and invokes the delegate.

C#

```
public void SimpleEmitDemo()  
{  
    DynamicMethod meth = new DynamicMethod("", null, null);  
    ILGenerator il = meth.GetILGenerator();  
    il.EmitWriteLine("Hello, World!");  
    il.Emit(OpCodes.Ret);  
  
    Test1 t1 = (Test1) meth.CreateDelegate(typeof(Test1));  
    t1();  
}
```

3. In your main program, get the display name of your assembly. This name is used when you create instances of the `Worker` class in the sandboxed application domain.

C#

```
String asmName = typeof(Worker).Assembly.FullName;
```

4. In your main program, create a sandboxed application domain, as described in [the first procedure](#) in this walkthrough. You do not have to add any permissions to the `Internet` permission set, because the `SimpleEmitDemo` method uses only public methods.
5. In your main program, create an instance of the `Worker` class in the sandboxed application domain.

C#

```
Worker w = (Worker) ad.CreateInstanceAndUnwrap(asmName, "Worker");
```

The `CreateInstanceAndUnwrap` method creates the object in the target application domain and returns a proxy that can be used to call the properties and methods of the object.

ⓘ Note

If you use this code in Visual Studio, you must change the name of the class to include the namespace. By default, the namespace is the name of the project. For example, if the project is "PartialTrust", the class name must be "PartialTrust.Worker".

6. Add code to call the `SimpleEmitDemo` method. The call is marshalled across the application domain boundary, and the code is executed in the sandboxed application domain.

C#

```
w.SimpleEmitDemo();
```

Using Anonymously Hosted Dynamic Methods

Anonymously hosted dynamic methods are associated with a transparent assembly that is provided by the system. Therefore, the code they contain is transparent. Ordinary dynamic methods, on the other hand, must be associated with an existing module

(whether directly specified or inferred from an associated type), and take their security level from that module.

ⓘ Note

The only way to associate a dynamic method with the assembly that provides anonymous hosting is to use the constructors that are described in the following procedure. You cannot explicitly specify a module in the anonymous hosting assembly.

Ordinary dynamic methods have access to the internal members of the module they are associated with, or to the private members of the type they are associated with. Because anonymously hosted dynamic methods are isolated from other code, they do not have access to private data. However, they do have a restricted ability to skip JIT visibility checks to gain access to private data. This ability is limited to assemblies that have trust levels equal to or less than the trust level of the assembly that emits the code.

To prevent elevation of privilege, stack information for the emitting assembly is included when anonymously hosted dynamic methods are constructed. When the method is invoked, the stack information is checked. An anonymously hosted dynamic method that is invoked from fully trusted code is still limited to the trust level of the assembly that emitted it.

To use anonymously hosted dynamic methods

- Create an anonymously hosted dynamic method by using a constructor that does not specify an associated module or type.

C#

```
DynamicMethod meth = new DynamicMethod("", null, null);  
ILGenerator il = meth.GetILGenerator();  
il.EmitWriteLine("Hello, World!");  
il.Emit(OpCodes.Ret);
```

If an anonymously hosted dynamic method uses only public types and methods, it does not require restricted member access and does not have to skip JIT visibility checks.

No special permissions are required to emit a dynamic method, but the emitted code requires the permissions that are demanded by the types and methods it uses. For example, if the emitted code calls a method that accesses a file, it

requires [FileIOPermission](#). If the trust level does not include that permission, a security exception is thrown when the emitted code is executed. The code shown here emits a dynamic method that uses only the [Console.WriteLine](#) method. Therefore, the code can be executed from partially trusted locations.

- Alternatively, create an anonymously hosted dynamic method with restricted ability to skip JIT visibility checks, by using the [DynamicMethod\(String, Type, Type\[\], Boolean\)](#) constructor and specifying `true` for the `restrictedSkipVisibility` parameter.

```
C#  
  
DynamicMethod meth = new DynamicMethod("",  
                                     typeof(char),  
                                     new Type[] { typeof(String) },  
                                     true);
```

The restriction is that the anonymously hosted dynamic method can access private data only in assemblies with trust levels equal to or less than the trust level of the emitting assembly. For example, if the dynamic method is executing with Internet trust, it can access private data in other assemblies that are also executing with Internet trust, but it cannot access private data of .NET Framework assemblies. .NET Framework assemblies are installed in the global assembly cache and are always fully trusted.

Anonymously hosted dynamic methods can use this restricted ability to skip JIT visibility checks only if the host application grants [ReflectionPermission](#) with the [ReflectionPermissionFlag.RestrictedMemberAccess](#) flag. The demand for this permission is made when the method is invoked.

⚠ Note

Call stack information for the emitting assembly is included when the dynamic method is constructed. Therefore, the demand is made against the permissions of the emitting assembly instead of the assembly that invokes the method. This prevents the emitted code from being executed with elevated permissions.

The [complete code example](#) at the end of this walkthrough demonstrates the use and limitations of restricted member access. Its `Worker` class includes a method that can create anonymously hosted dynamic methods with or without the

restricted ability to skip visibility checks, and the example shows the result of executing this method in application domains that have different trust levels.

⚠ **Note**

The restricted ability to skip visibility checks is a feature of anonymously hosted dynamic methods. When ordinary dynamic methods skip JIT visibility checks, they must be granted full trust.

Example

Description

The following code example demonstrates the use of the [RestrictedMemberAccess](#) flag to allow anonymously hosted dynamic methods to skip JIT visibility checks, but only when the target member is at an equal or lower level of trust than the assembly that emits the code.

The example defines a `Worker` class that can be marshalled across application domain boundaries. The class has two `AccessPrivateMethod` method overloads that emit and execute dynamic methods. The first overload emits a dynamic method that calls the private `PrivateMethod` method of the `Worker` class, and it can emit the dynamic method with or without JIT visibility checks. The second overload emits a dynamic method that accesses an `internal` property (`Friend` property in Visual Basic) of the [String](#) class.

The example uses a helper method to create a grant set limited to `Internet` permissions, and then creates an application domain, using the [AppDomain.CreateDomain\(String, Evidence, AppDomainSetup, PermissionSet, StrongName\[\]\)](#) method overload to specify that all code that executes in the domain uses this grant set. The example creates an instance of the `Worker` class in the application domain, and executes the `AccessPrivateMethod` method two times.

- The first time the `AccessPrivateMethod` method is executed, JIT visibility checks are enforced. The dynamic method fails when it is invoked, because JIT visibility checks prevent it from accessing the private method.
- The second time the `AccessPrivateMethod` method is executed, JIT visibility checks are skipped. The dynamic method fails when it is compiled, because the `Internet` grant set does not grant sufficient permissions to skip visibility checks.

The example adds [ReflectionPermission](#) with [ReflectionPermissionFlag.RestrictedMemberAccess](#) to the grant set. The example then creates a second domain, specifying that all code that executes in the domain is granted the permissions in the new grant set. The example creates an instance of the `Worker` class in the new application domain, and executes both overloads of the `AccessPrivateMethod` method.

- The first overload of the `AccessPrivateMethod` method is executed, and JIT visibility checks are skipped. The dynamic method compiles and executes successfully, because the assembly that emits the code is the same as the assembly that contains the private method. Therefore, the trust levels are equal. If the application that contains the `Worker` class had several assemblies, the same process would succeed for any one of those assemblies, because they would all be at the same trust level.
- The second overload of the `AccessPrivateMethod` method is executed, and again JIT visibility checks are skipped. This time the dynamic method fails when it is compiled, because it tries to access the `internal FirstChar` property of the [String](#) class. The assembly that contains the [String](#) class is fully trusted. Therefore, it is at a higher level of trust than the assembly that emits the code.

This comparison shows how [ReflectionPermissionFlag.RestrictedMemberAccess](#) enables partially trusted code to skip visibility checks for other partially trusted code without compromising the security of trusted code.

Code

C#

```
using System;
using System.Reflection.Emit;
using System.Reflection;
using System.Security;
using System.Security.Permissions;
using System.Security.Policy;
using System.Collections;
using System.Diagnostics;

// This code example works properly only if it is run from a fully
// trusted location, such as your local computer.

// Delegates used to execute the dynamic methods.
//
public delegate void Test(Worker w);
public delegate void Test1();
public delegate char Test2(String instance);
```

```

// The Worker class must inherit MarshalByRefObject so that its public
// methods can be invoked across application domain boundaries.
//
public class Worker : MarshalByRefObject
{
    private void PrivateMethod()
    {
        Console.WriteLine("Worker.PrivateMethod()");
    }

    public void SimpleEmitDemo()
    {
        DynamicMethod meth = new DynamicMethod("", null, null);
        ILGenerator il = meth.GetILGenerator();
        il.EmitWriteLine("Hello, World!");
        il.Emit(OpCodes.Ret);

        Test1 t1 = (Test1) meth.CreateDelegate(typeof(Test1));
        t1();
    }

    // This overload of AccessPrivateMethod emits a dynamic method and
    // specifies whether to skip JIT visibility checks. It creates a
    // delegate for the method and invokes the delegate. The dynamic
    // method calls a private method of the Worker class.
    public void AccessPrivateMethod(bool restrictedSkipVisibility)
    {
        // Create an unnamed dynamic method that has no return type,
        // takes one parameter of type Worker, and optionally skips JIT
        // visibility checks.
        DynamicMethod meth = new DynamicMethod(
            "",
            null,
            new Type[] { typeof(Worker) },
            restrictedSkipVisibility);

        // Get a MethodInfo for the private method.
        MethodInfo pvtMeth = typeof(Worker).GetMethod("PrivateMethod",
            BindingFlags.NonPublic | BindingFlags.Instance);

        // Get an ILGenerator and emit a body for the dynamic method.
        ILGenerator il = meth.GetILGenerator();

        // Load the first argument, which is the target instance, onto the
        // execution stack, call the private method, and return.
        il.Emit(OpCodes.Ldarg_0);
        il.EmitCall(OpCodes.Call, pvtMeth, null);
        il.Emit(OpCodes.Ret);

        // Create a delegate that represents the dynamic method, and
        // invoke it.
        try
        {
            Test t = (Test) meth.CreateDelegate(typeof(Test));

```

```

        try
        {
            t(this);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"{ex.GetType().Name} was thrown when the
delegate was invoked.");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"{ex.GetType().Name} was thrown when the
delegate was compiled.");
    }
}

// This overload of AccessPrivateMethod emits a dynamic method that
takes
// a string and returns the first character, using a private field of
the
// String class. The dynamic method skips JIT visibility checks.
public void AccessPrivateMethod()
{
    DynamicMethod meth = new DynamicMethod("",
                                           typeof(char),
                                           new Type[] { typeof(String)
},
                                           true);

    // Get a MethodInfo for the 'get' accessor of the private property.
    PropertyInfo pi = typeof(System.String).GetProperty(
        "FirstChar",
        BindingFlags.NonPublic | BindingFlags.Instance);
    MethodInfo pvtMeth = pi.GetGetMethod(true);

    // Get an ILGenerator and emit a body for the dynamic method.
    ILGenerator il = meth.GetILGenerator();

    // Load the first argument, which is the target string, onto the
    // execution stack, call the 'get' accessor to put the result onto
    // the execution stack, and return.
    il.Emit(OpCodes.Ldarg_0);
    il.EmitCall(OpCodes.Call, pvtMeth, null);
    il.Emit(OpCodes.Ret);

    // Create a delegate that represents the dynamic method, and
    // invoke it.
    try
    {
        Test2 t = (Test2) meth.CreateDelegate(typeof(Test2));
        char first = t("Hello, World!");
        Console.WriteLine($"{first} is the first character.");
    }
    catch (Exception ex)

```

```

    {
        Console.WriteLine($"{ex.GetType().Name} was thrown when the
delegate was compiled.");
    }
}

// The entry point for the code example.
static void Main()
{
    // Get the display name of the executing assembly, to use when
    // creating objects to run code in application domains.
    String asmName = typeof(Worker).Assembly.FullName;

    // Create the permission set to grant to other assemblies. In this
    // case they are the permissions found in the Internet zone.
    Evidence ev = new Evidence();
    ev.AddHostEvidence(new Zone(SecurityZone.Internet));
    PermissionSet pset = new NamedPermissionSet("Internet",
SecurityManager.GetStandardSandbox(ev));

    // For simplicity, set up the application domain to use the
    // current path as the application folder, so the same executable
    // can be used in both trusted and untrusted scenarios. Normally
    // you would not do this with real untrusted code.
    AppDomainSetup adSetup = new AppDomainSetup();
    adSetup.ApplicationBase = ".";

    // Create an application domain in which all code that executes is
    // granted the permissions of an application run from the Internet.
    AppDomain ad = AppDomain.CreateDomain("Sandbox", ev, adSetup, pset,
null);

    // Create an instance of the Worker class in the partially trusted
    // domain. Note: If you build this code example in Visual Studio,
    // you must change the name of the class to include the default
    // namespace, which is the project name. For example, if the project
    // is "AnonymouslyHosted", the class is "AnonymouslyHosted.Worker".
    Worker w = (Worker) ad.CreateInstanceAndUnwrap(asmName, "Worker");

    // Emit a simple dynamic method that prints "Hello, World!"
    w.SimpleEmitDemo();

    // Emit and invoke a dynamic method that calls a private method
    // of Worker, with JIT visibility checks enforced. The call fails
    // when the delegate is invoked.
    w.AccessPrivateMethod(false);

    // Emit and invoke a dynamic method that calls a private method
    // of Worker, skipping JIT visibility checks. The call fails when
    // the method is invoked.
    w.AccessPrivateMethod(true);

    // Unload the application domain. Add RestrictedMemberAccess to the
    // grant set, and use it to create an application domain in which
    // partially trusted code can call private members, as long as the

```

```

// trust level of those members is equal to or lower than the trust
// level of the partially trusted code.
AppDomain.Unload(ad);
pset.SetPermission(
    new ReflectionPermission(
        ReflectionPermissionFlag.RestrictedMemberAccess));
ad = AppDomain.CreateDomain("Sandbox2", ev, adSetup, pset, null);

// Create an instance of the Worker class in the partially trusted
// domain.
w = (Worker) ad.CreateInstanceAndUnwrap(asmName, "Worker");

// Again, emit and invoke a dynamic method that calls a private
method
// of Worker, skipping JIT visibility checks. This time compilation
// succeeds because of the grant for RestrictedMemberAccess.
w.AccessPrivateMethod(true);

// Finally, emit and invoke a dynamic method that calls an internal
// method of the String class. The call fails, because the trust
level
// of the assembly that contains String is higher than the trust
level
// of the assembly that emits the dynamic method.
w.AccessPrivateMethod();
}
}

/* This code example produces the following output:

Hello, World!
MethodAccessException was thrown when the delegate was invoked.
MethodAccessException was thrown when the delegate was invoked.
Worker.PrivateMethod()
MethodAccessException was thrown when the delegate was compiled.
*/

```

Compiling the Code

- If you build this code example in Visual Studio, you must change the name of the class to include the namespace when you pass it to the [CreateInstanceAndUnwrap](#) method. By default, the namespace is the name of the project. For example, if the project is "PartialTrust", the class name must be "PartialTrust.Worker".

See also

- [Security Issues in Reflection Emit](#)
- [How to: Run Partially Trusted Code in a Sandbox](#)

Dynamic language runtime overview

Article • 03/30/2024

The *dynamic language runtime* (DLR) is a runtime environment that adds a set of services for dynamic languages to the common language runtime (CLR). The DLR makes it easier to develop dynamic languages to run on .NET and to add dynamic features to statically typed languages.

Dynamic languages can identify the type of an object at run time, whereas in statically typed languages such as C# and Visual Basic (when you use `Option Explicit On`), you must specify object types at design time. Examples of dynamic languages are Lisp, Smalltalk, JavaScript, PHP, Ruby, Python, ColdFusion, Lua, Cobra, and Groovy.

Most dynamic languages provide the following advantages for developers:

- The ability to use a rapid feedback loop (REPL, or read-evaluate-print loop). This lets you enter several statements and immediately execute them to see the results.
- Support for both top-down development and more traditional bottom-up development. For example, when you use a top-down approach, you can call functions that aren't yet implemented and then add underlying implementations when you need them.
- Easier refactoring and code modifications, because you don't have to change static type declarations throughout the code.

Dynamic languages make excellent scripting languages. Customers can easily extend applications created by using dynamic languages with new commands and functionality. Dynamic languages are also frequently used for creating web sites and test harnesses, maintaining server farms, developing various utilities, and performing data transformations.

The purpose of the DLR is to enable a system of dynamic languages to run on .NET and give them .NET interoperability. The DLR adds dynamic objects to C# and Visual Basic to support dynamic behavior in these languages and enable their interoperation with dynamic languages.

The DLR also helps you create libraries that support dynamic operations. For example, if you have a library that uses XML or JavaScript Object Notation (JSON) objects, your objects can appear as dynamic objects to languages that use the DLR. This lets library users write syntactically simpler and more natural code for operating with objects and accessing object members.

For example, you might use the following code to increment a counter in XML in C#.


```
Scriptobj.SetProperty("Count", ((int)GetProperty("Count")) + 1);
```

By using the DLR, you could use the following code instead for the same operation.

```
scriptobj.Count += 1;
```

Like the CLR, the DLR is a part of .NET. It's available for download on the [IronLanguages/dlr](#) repo on GitHub.

[IronPython](#) is an example of a language that was developed by using the DLR.

Primary DLR advantages

The DLR provides the following advantages.

Simplifies porting dynamic languages to .NET

The DLR allows language implementers to avoid creating lexical analyzers, parsers, semantic analyzers, code generators, and other tools that they traditionally had to create themselves. To use the DLR, a language needs to produce *expression trees*, which represent language-level code in a tree-shaped structure, runtime helper routines, and optional dynamic objects that implement the [IDynamicMetaObjectProvider](#) interface. The DLR and .NET automate a lot of code analysis and code generation tasks. This enables language implementers to concentrate on unique language features.

Enables dynamic features in statically typed languages

Existing .NET languages such as C# and Visual Basic can create dynamic objects and use them together with statically typed objects. For example, C# and Visual Basic can use dynamic objects for HTML, Document Object Model (DOM), and reflection.

Provides future benefits of the DLR and .NET

Languages implemented by using the DLR can benefit from future DLR and .NET improvements. For example, if .NET releases a new version that has an improved garbage collector or faster assembly loading time, languages implemented by using the DLR immediately get the same benefit. If the DLR adds optimizations such as better compilation, the performance also improves for all languages implemented by using the DLR.

Enables sharing of libraries and objects

The objects and libraries implemented in one language can be used by other languages. The DLR also enables interoperation between statically typed and dynamic languages. For example, C# can declare a dynamic object that uses a library that is written in a dynamic language. At the same time, dynamic languages can use libraries from the .NET Framework.

Provides fast dynamic dispatch and invocation

The DLR provides fast execution of dynamic operations by supporting advanced polymorphic caching. The DLR creates rules for binding operations that use objects to the necessary runtime implementations and then caches these rules to avoid resource-exhausting binding computations during successive executions of the same code on the same types of objects.

DLR architecture

The DLR adds a set of services to the CLR for better supporting dynamic languages. These services include the following:

- Expression trees. The DLR uses expression trees to represent language semantics. For this purpose, the DLR has extended LINQ expression trees to include control flow, assignment, and other language-modeling nodes. For more information, see [Expression Trees \(C#\)](#) or [Expression Trees \(Visual Basic\)](#).
- Call site caching. A *dynamic call site* is a place in the code where you perform an operation like `a + b` or `a.b()` on dynamic objects. The DLR caches the characteristics of `a` and `b` (usually the types of these objects) and information about the operation. If such an operation has been performed previously, the DLR retrieves all the necessary information from the cache for fast dispatch.
- Dynamic object interoperability. The DLR provides a set of classes and interfaces that represent dynamic objects and operations and can be used by language implementers and authors of dynamic libraries. These classes and interfaces include [IDynamicMetaObjectProvider](#), [DynamicMetaObject](#), [DynamicObject](#), and [ExpandoObject](#).

The DLR uses binders in call sites to communicate not only with .NET, but with other infrastructures and services, such as COM. Binders encapsulate a language's semantics and specify how to perform operations in a call site by using expression trees. This enables dynamic and statically typed languages that use the DLR to share libraries and gain access to all the technologies that the DLR supports.

DLR documentation

For more information about how to use the open source version of the DLR to add dynamic behavior to a language, or about how to enable the use of a dynamic language with .NET, see the documentation on the [IronLanguages/dlr](#) repo on GitHub.

See also

- [ExpandoObject](#)
- [DynamicObject](#)
- [Common Language Runtime](#)
- [Expression Trees \(C#\)](#)
- [Expression Trees \(Visual Basic\)](#)
- [Walkthrough: Create and Use Dynamic Objects](#)

Compile and generate dynamic source code

Article • 03/30/2024

.NET includes a mechanism called the Code Document Object Model (CodeDOM) that enables developers of programs that emit source code to generate source code in multiple programming languages at run time, based on a single model that represents the code to render.

To represent source code, CodeDOM elements are linked to each other to form a data structure known as a CodeDOM graph, which models the structure of some source code.

The [System.CodeDom](#) namespace defines types that can represent the logical structure of source code, independent of a specific programming language. The [System.CodeDom.Compiler](#) namespace defines types for generating source code from CodeDOM graphs and managing the compilation of source code in supported languages. Compiler vendors or developers can extend the set of supported languages.

Language-independent source code modeling can be valuable when a program needs to generate source code for a program model in multiple languages or for an uncertain target language. For example, some designers use the CodeDOM as a language abstraction interface to produce source code in the correct programming language, if CodeDOM support for the language is available.

.NET includes code generators and code compilers for [CSharpCodeProvider](#), [JScriptCodeProvider](#), and [VBCodeProvider](#).

Reference

- [System.CodeDom](#)

Defines elements that represent code elements in programming languages that target the common language runtime.

- [System.CodeDom.Compiler](#)

Defines interfaces for generating and compiling code at run time.

Related sections

- [CodeDOM Quick Reference](#) provides a quick way for developers to find the CodeDOM elements that represent source code elements.

Use the CodeDOM

Article • 03/30/2024

The CodeDOM provides types that represent many common types of source code elements. You can design a program that builds a source code model using CodeDOM elements to assemble an object graph. This object graph can be rendered as source code using a CodeDOM code generator for a supported programming language. The CodeDOM can also be used to compile source code into a binary assembly.

Some common uses for the CodeDOM include:

- Templated code generation: generating code for ASP.NET, XML Web services client proxies, code wizards, designers, or other code-emitting mechanisms.
- Dynamic compilation: supporting code compilation in single or multiple languages.

Build a CodeDOM graph

The [System.CodeDom](#) namespace provides classes for representing the logical structure of source code, independent of language syntax.

The structure of a CodeDOM graph

The structure of a CodeDOM graph is like a tree of containers. The top-most, or root, container of each compilable CodeDOM graph is a [CodeCompileUnit](#). Every element of your source code model must be linked into the graph through a property of a [CodeObject](#) in the graph.

Build a source code model for a sample Hello World program

The following walkthrough provides an example of how to build a CodeDOM object graph that represents the code for a simple Hello World application. For the complete source code for this code example, see the [System.CodeDom.Compiler.CodeDomProvider](#) article.

Create a compile unit

The CodeDOM defines an object called a [CodeCompileUnit](#), which can reference a CodeDOM object graph that models the source code to compile. A **CodeCompileUnit** has properties for storing references to attributes, namespaces, and assemblies.

The CodeDom providers that derive from the [CodeDomProvider](#) class contain methods that process the object graph referenced by a **CodeCompileUnit**.

To create an object graph for a simple application, you must assemble the source code model and reference it from a **CodeCompileUnit**.

You can create a new compile unit with the syntax demonstrated in this example:

C#

```
CodeCompileUnit compileUnit = new CodeCompileUnit();
```

A [CodeSnippetCompileUnit](#) can contain a section of source code that's already in the target language, but cannot be rendered to another language.

Define a namespace

To define a namespace, create a [CodeNamespace](#) and assign a name for it using the appropriate constructor or by setting its **Name** property.

C#

```
CodeNamespace samples = new CodeNamespace("Samples");
```

Import a namespace

To add a namespace import directive to the namespace, add a [CodeNamespaceImport](#) that indicates the namespace to import to the **CodeNamespace.Imports** collection.

The following code adds an import for the **System** namespace to the **Imports** collection of a **CodeNamespace** named `samples`:

C#

```
samples.Imports.Add(new CodeNamespaceImport("System"));
```

Link code elements into the object graph

All code elements that form a CodeDOM graph must be linked to the [CodeCompileUnit](#) that is the root element of the tree by a series of references between elements directly referenced from the properties of the root object of the graph. Set an object to a property of a container object to establish a reference from the container object.

The following statement adds the `samples` **CodeNamespace** to the **Namespaces** collection property of the root **CodeCompileUnit**.

C#

```
compileUnit.Namespaces.Add( samples );
```

Define a type

To declare a class, structure, interface, or enumeration using the CodeDOM, create a new [CodeTypeDeclaration](#), and assign it a name. The following example demonstrates this using a constructor overload to set the **Name** property:

C#

```
CodeTypeDeclaration class1 = new CodeTypeDeclaration("Class1");
```

To add a type to a namespace, add a [CodeTypeDeclaration](#) that represents the type to add to the namespace to the **Types** collection of a **CodeNamespace**.

The following example demonstrates how to add a class named `class1` to a **CodeNamespace** named `samples`:

C#

```
samples.Types.Add(class1);
```

Add class members to a class

The [System.CodeDom](#) namespace provides a variety of elements that can be used to represent class members. Each class member can be added to the **Members** collection of a [CodeTypeDeclaration](#).

Define a code entry point method for an executable

If you are building code for an executable program, it is necessary to indicate the entry point of a program by creating a [CodeEntryPointMethod](#) to represent the method at which program execution should begin.

The following example demonstrates how to define an entry point method that contains a [CodeMethodInvokeExpression](#) that calls **System.Console.WriteLine** to print "Hello

World!":

C#

```
CodeEntryPointMethod start = new CodeEntryPointMethod();
CodeMethodInvokeExpression cs1 = new CodeMethodInvokeExpression(
    new CodeTypeReferenceExpression("System.Console"),
    "WriteLine", new CodePrimitiveExpression("Hello World!"));
start.Statements.Add(cs1);
```

The following statement adds the entry point method named `start` to the **Members** collection of `class1`:

C#

```
class1.Members.Add( start );
```

Now the `CodeCompileUnit` named `compileUnit` contains the CodeDOM graph for a simple Hello World program. For information on generating and compiling code from a CodeDOM graph, see [Generating Source Code and Compiling a Program from a CodeDOM Graph](#).

More information on building a CodeDOM graph

The CodeDOM supports the many common types of code elements found in programming languages that support the common language runtime. The CodeDOM was not designed to provide elements to represent all possible programming language features. Code that cannot be represented easily with CodeDOM elements can be encapsulated in a [CodeSnippetExpression](#), a [CodeSnippetStatement](#), a [CodeSnippetTypeMember](#), or a [CodeSnippetCompileUnit](#). However, snippets cannot be translated to other languages automatically by the CodeDOM.

For documentation for each of the CodeDOM types, see the reference documentation for the [System.CodeDom](#) namespace.

For a quick chart to locate the CodeDOM element that represents a specific type of code element, see the [CodeDOM Quick Reference](#).

Generate and compile source code from a CodeDOM graph

Article • 03/27/2024

The [System.CodeDom.Compiler](#) namespace provides interfaces for generating source code from CodeDOM object graphs and for managing compilation with supported compilers. A code provider can produce source code in a particular programming language according to a CodeDOM graph. A class that derives from [CodeDomProvider](#) can typically provide methods for generating and compiling code for the language the provider supports.

Use a CodeDOM code provider to generate source code

To generate source code in a particular language, you need a CodeDOM graph that represents the structure of the source code to generate.

The following example demonstrate how to create an instance of a [CSharpCodeProvider](#):

C#

```
CSharpCodeProvider provider = new CSharpCodeProvider();
```

The graph for code generation is typically contained in a [CodeCompileUnit](#). To generate code for a `CodeCompileUnit` that contains a CodeDOM graph, call the [GenerateCodeFromCompileUnit](#) method of the code provider. This method has a parameter for a [TextWriter](#) that it uses to generate the source code, so it is sometimes necessary to first create a `TextWriter` that can be written to. The following example demonstrates generating code from a `CodeCompileUnit` and writing the generated source code to a file named HelloWorld.cs.

C#

```
public static string GenerateCSharpCode(CodeCompileUnit compileunit)
{
    // Generate the code with the C# code provider.
    CSharpCodeProvider provider = new CSharpCodeProvider();

    // Build the output file name.
    string sourceFile;
```

```

if (provider.FileExtension[0] == '.')
{
    sourceFile = "HelloWorld" + provider.FileExtension;
}
else
{
    sourceFile = "HelloWorld." + provider.FileExtension;
}

// Create a TextWriter to a StreamWriter to the output file.
using (StreamWriter sw = new StreamWriter(sourceFile, false))
{
    IndentedTextWriter tw = new IndentedTextWriter(sw, "    ");

    // Generate source code using the code provider.
    provider.GenerateCodeFromCompileUnit(compileunit, tw,
        new CodeGeneratorOptions());

    // Close the output file.
    tw.Close();
}

return sourceFile;
}

```

Use a CodeDOM code provider to compile assemblies

Invoke compilation

To compile an assembly using a CodeDom provider, you must have either source code to compile in a language for which you have a compiler, or a CodeDOM graph that source code to compile can be generated from.

If you are compiling from a CodeDOM graph, pass the [CodeCompileUnit](#) containing the graph to the [CompileAssemblyFromDom](#) method of the code provider. If you have a source code file in a language that the compiler understands, pass the name of the file containing the source code to the [CompileAssemblyFromFile](#) method of the CodeDom provider. You can also pass a string containing source code in a language that the compiler understands to the [CompileAssemblyFromSource](#) method of the CodeDom provider.

Configure compilation parameters

All of the standard compilation-invoking methods of a CodeDom provider have a parameter of type [CompilerParameters](#) that indicates the options to use for compilation.

You can specify a file name for the output assembly in the [OutputAssembly](#) property of the [CompilerParameters](#). Otherwise, a default output file name will be used.

By default, a new [CompilerParameters](#) is initialized with its [GenerateExecutable](#) property set to `false`. If you are compiling an executable program, you must set the [GenerateExecutable](#) property to `true`. When the [GenerateExecutable](#) is set to `false`, the compiler will generate a class library.

If you are compiling an executable from a CodeDOM graph, a [CodeEntryPointMethod](#) must be defined in the graph. If there are multiple code entry points, it may be necessary to set the [MainClass](#) property of the [CompilerParameters](#) to the name of the class that defines the entry point to use.

To include debug information in a generated executable, set the [IncludeDebugInformation](#) property to `true`.

If your project references any assemblies, you must specify the assembly names as items in a [StringCollection](#) as the [ReferencedAssemblies](#) property of the [CompilerParameters](#) you use when invoking compilation.

You can compile an assembly that is written to memory rather than disk by setting the [GenerateInMemory](#) property to `true`. When an assembly is generated in memory, your code can obtain a reference to the generated assembly from the [CompiledAssembly](#) property of a [CompilerResults](#). If an assembly is written to disk, you can obtain the path to the generated assembly from the [PathToAssembly](#) property of a [CompilerResults](#).

To specify a custom command-line arguments string to use when invoking the compilation process, set the string in the [CompilerOptions](#) property.

If a Win32 security token is required to invoke the compiler process, specify the token in the [UserToken](#) property.

To link a Win32 resource file into the compiled assembly, specify the name of the Win32 resource file in the [Win32Resource](#) property.

To specify a warning level at which to halt compilation, set the [WarningLevel](#) property to an integer that represents the warning level at which to halt compilation. You can also configure the compiler to halt compilation if warnings are encountered by setting the [TreatWarningsAsErrors](#) property to `true`.

The following code example demonstrates compiling a source file using a CodeDom provider derived from the [CodeDomProvider](#) class.

C#

```
public static bool CompileCSharpCode(string sourceFile, string exeFile)
{
    CSharpCodeProvider provider = new CSharpCodeProvider();

    // Build the parameters for source compilation.
    CompilerParameters cp = new CompilerParameters();

    // Add an assembly reference.
    cp.ReferencedAssemblies.Add( "System.dll" );

    // Generate an executable instead of
    // a class library.
    cp.GenerateExecutable = true;

    // Set the assembly file name to generate.
    cp.OutputAssembly = exeFile;

    // Save the assembly as a physical file.
    cp.GenerateInMemory = false;

    // Invoke compilation.
    CompilerResults cr = provider.CompileAssemblyFromFile(cp, sourceFile);

    if (cr.Errors.Count > 0)
    {
        // Display compilation errors.
        Console.WriteLine($"Errors building {sourceFile} into
{cr.PathToAssembly}");
        foreach (CompilerError ce in cr.Errors)
        {
            Console.WriteLine($" {ce.ToString()}");
            Console.WriteLine();
        }
    }
    else
    {
        Console.WriteLine($"Source {sourceFile} built into
{cr.PathToAssembly} successfully.");
    }

    // Return the results of compilation.
    if (cr.Errors.Count > 0)
    {
        return false;
    }
    else
    {
        return true;
    }
}
```

```
}  
}
```

Languages with initial support

.NET provides code compilers and code generators for the following languages: C#, Visual Basic, C++, and JScript. CodeDOM support can be extended to other languages by implementing language-specific code generators and code compilers.

See also

- [System.CodeDom](#)
- [System.CodeDom.Compiler](#)
- [Dynamic Source Code Generation and Compilation](#)
- [CodeDOM Quick Reference](#)

How to: Create an XML documentation file using CodeDOM

Article • 09/23/2021

CodeDOM can be used to create code that generates XML documentation. The process involves creating the CodeDOM graph that contains the XML documentation comments, generating the code, and compiling the generated code with the compiler option that creates the XML documentation output.

Create a CodeDOM graph

1. Create a [CodeCompileUnit](#) containing the CodeDOM graph for the sample application.
2. Use the [CodeCommentStatement](#) constructor with the `docComment` parameter set to `true` to create the XML documentation comment elements and text.

C#

```
CodeTypeDeclaration class1 = new CodeTypeDeclaration("Class1");

class1.Comments.Add(new CodeCommentStatement("<summary>", true));
class1.Comments.Add(new CodeCommentStatement(
    "Create a Hello World application.", true));
class1.Comments.Add(new CodeCommentStatement("</summary>", true));
class1.Comments.Add(new CodeCommentStatement(
    @"<seealso cref=" + "'" + "Class1.Main" + "'" + "/>", true));

// Add the new type to the namespace type collection.
samples.Types.Add(class1);

// Declare a new code entry point method.
CodeEntryPointMethod start = new CodeEntryPointMethod();
start.Comments.Add(new CodeCommentStatement("<summary>", true));
start.Comments.Add(new CodeCommentStatement(
    "Main method for HelloWorld application.", true));
start.Comments.Add(new CodeCommentStatement(
    @"<para>Add a new paragraph to the description.</para>", true));
start.Comments.Add(new CodeCommentStatement("</summary>", true));
```

Generate the code from the CodeCompileUnit

Use the [GenerateCodeFromCompileUnit](#) method to generate the code and create a source file to be compiled.

C#

```
StreamWriter sourceFile = new StreamWriter(sourceFileName);  
provider.GenerateCodeFromCompileUnit(cu, sourceFile, null);  
sourceFile.Close();
```

Compile the code and generate the documentation file

Add the `/doc` compiler option to the [CompilerOptions](#) property of a [CompilerParameters](#) object and pass the object to the [CompileAssemblyFromFile](#) method to create the XML documentation file when the code is compiled.

C#

```
CompilerParameters opt = new CompilerParameters(new string[] {  
    "System.dll" });  
opt.GenerateExecutable = true;  
opt.OutputAssembly = "HelloWorld.exe";  
opt.TreatWarningsAsErrors = true;  
opt.IncludeDebugInformation = true;  
opt.GenerateInMemory = true;  
opt.CompilerOptions = "/doc:HelloWorldDoc.xml";  
  
CompilerResults results;  
  
LogMessage("Compiling with " + providerName);  
results = provider.CompileAssemblyFromFile(opt, sourceFileName);
```

Example

The following code example creates a CodeDOM graph with documentation comments, generates a code file from the graph, and compiles the file and creates an associated XML documentation file.

C#

```
using System;  
using System.CodeDom;  
using System.CodeDom.Compiler;  
using System.IO;  
using System.Text.RegularExpressions;  
  
namespace BasicCodeDomApp  
{  
    class Program  
    {  
        static string providerName = "cs";
```



```

static string sourceFileName = "test.cs";
static void Main(string[] args)
{
    CodeDomProvider provider =
        CodeDomProvider.CreateProvider(providerName);

    LogMessage("Building CodeDOM graph...");

    CodeCompileUnit cu = new CodeCompileUnit();

    cu = BuildHelloWorldGraph();

    StreamWriter sourceFile = new StreamWriter(sourceFileName);
    provider.GenerateCodeFromCompileUnit(cu, sourceFile, null);
    sourceFile.Close();

    CompilerParameters opt = new CompilerParameters(new string[]{
        "System.dll" });
    opt.GenerateExecutable = true;
    opt.OutputAssembly = "HelloWorld.exe";
    opt.TreatWarningsAsErrors = true;
    opt.IncludeDebugInformation = true;
    opt.GenerateInMemory = true;
    opt.CompilerOptions = "/doc:HelloWorldDoc.xml";

    CompilerResults results;

    LogMessage("Compiling with " + providerName);
    results = provider.CompileAssemblyFromFile(opt, sourceFileName);

    OutputResults(results);
    if (results.NativeCompilerReturnValue != 0)
    {
        LogMessage("");
        LogMessage("Compilation failed.");
    }
    else
    {
        LogMessage("");
        LogMessage("Demo completed successfully.");
    }
    File.Delete(sourceFileName);
}

// Build a Hello World program graph using
// System.CodeDom types.
public static CodeCompileUnit BuildHelloWorldGraph()
{
    // Create a new CodeCompileUnit to contain
    // the program graph.
    CodeCompileUnit compileUnit = new CodeCompileUnit();

    // Declare a new namespace called Samples.
    CodeNamespace samples = new CodeNamespace("Samples");
    // Add the new namespace to the compile unit.

```

```

compileUnit.Namespaces.Add(samples);

// Add the new namespace import for the System namespace.
samples.Imports.Add(new CodeNamespaceImport("System"));

// Declare a new type called Class1.
CodeTypeDeclaration class1 = new CodeTypeDeclaration("Class1");

class1.Comments.Add(new CodeCommentStatement("<summary>",
true));
class1.Comments.Add(new CodeCommentStatement(
    "Create a Hello World application.", true));
class1.Comments.Add(new CodeCommentStatement("</summary>",
true));
class1.Comments.Add(new CodeCommentStatement(
    @"<seealso cref=" + "'" + "Class1.Main" + "'" + ">",
true));

// Add the new type to the namespace type collection.
samples.Types.Add(class1);

// Declare a new code entry point method.
CodeEntryPointMethod start = new CodeEntryPointMethod();
start.Comments.Add(new CodeCommentStatement("<summary>", true));
start.Comments.Add(new CodeCommentStatement(
    "Main method for HelloWorld application.", true));
start.Comments.Add(new CodeCommentStatement(
    @"<para>Add a new paragraph to the description.</para>",
true));
start.Comments.Add(new CodeCommentStatement("</summary>",
true));

// Create a type reference for the System.Console class.
CodeTypeReferenceExpression csSystemConsoleType =
    new CodeTypeReferenceExpression("System.Console");

// Build a Console.WriteLine statement.
CodeMethodInvokeExpression cs1 = new CodeMethodInvokeExpression(
    csSystemConsoleType, "WriteLine",
    new CodePrimitiveExpression("Hello World!"));

// Add the WriteLine call to the statement collection.
start.Statements.Add(cs1);

// Build another Console.WriteLine statement.
CodeMethodInvokeExpression cs2 = new CodeMethodInvokeExpression(
    csSystemConsoleType, "WriteLine", new
CodePrimitiveExpression(
    "Press the ENTER key to continue."));

// Add the WriteLine call to the statement collection.
start.Statements.Add(cs2);

// Build a call to System.Console.ReadLine.
CodeMethodInvokeExpression csReadLine =

```

```

        new CodeMethodInvokeExpression(csSystemConsoleType,
"ReadLine");

        // Add the ReadLine statement.
        start.Statements.Add(csReadLine);

        // Add the code entry point method to
        // the Members collection of the type.
        class1.Members.Add(start);

        return compileUnit;
    }
    static void LogMessage(string text)
    {
        Console.WriteLine(text);
    }

    static void OutputResults(CompilerResults results)
    {
        LogMessage("NativeCompilerReturnValue=" +
            results.NativeCompilerReturnValue.ToString());
        foreach (string s in results.Output)
        {
            LogMessage(s);
        }
    }
}

```

The code example creates the following XML documentation in the *HelloWorldDoc.xml* file.

XML

```

<?xml version="1.0" ?>
<doc>
  <assembly>
    <name>HelloWorld</name>
  </assembly>
  <members>
    <member name="T:Samples.Class1">
      <summary>
        Create a Hello World application.
      </summary>
      <seealso cref="M:Samples.Class1.Main" />
    </member>
    <member name="M:Samples.Class1.Main">
      <summary>
        Main method for HelloWorld application.
        <para>Add a new paragraph to the description.</para>
      </summary>
    </member>
  </members>
</doc>

```

```
</members>  
</doc>
```

Compile permissions

This code example requires the `FullTrust` permission set to execute successfully.

See also

- [Document your code with XML \(Visual Basic\)](#)
- [XML documentation comments](#)

How to: Create a class Using CodeDOM

Article • 09/15/2021

The following procedures illustrate how to create and compile a CodeDOM graph that generates a class containing two fields, three properties, a method, a constructor, and an entry point.

1. Create a console application that will use CodeDOM code to generate the source code for a class.

In this example, the generating class is named `Sample`, and the generated code is a class named `CodeDOMCreatedClass` in a file named `SampleCode`.

2. In the generating class, initialize the CodeDOM graph and use CodeDOM methods to define the members, constructor, and entry point (`Main` method) of the generated class.

In this example, the generated class has two fields, three properties, a constructor, a method, and a `Main` method.

3. In the generating class, create a language-specific code provider and call its [GenerateCodeFromCompileUnit](#) method to generate the code from the graph.
4. Compile and execute the application to generate the code.

In this example, the generated code is in a file named `SampleCode`. Compile and execute that code to see the sample output.

Create the application that will execute the CodeDOM code

Create a console application class to contain the CodeDOM code. Define the global fields that are to be used in the class to reference the assembly ([CodeCompileUnit](#)) and class ([CodeTypeDeclaration](#)), specify the name of the generated source file, and declare the `Main` method.

C#

```
using System;
using System.Reflection;
using System.IO;
using System.CodeDom;
using System.CodeDom.Compiler;
```

```
using Microsoft.CSharp;

namespace SampleCodeDom
{
    class Sample
    {
        CodeCompileUnit targetUnit;
        CodeTypeDeclaration targetClass;
        private const string outputFileName = "SampleCode.cs";
        static void Main(string[] args)
        {
        }
    }
}
```

Initialize the CodeDOM graph

In the constructor for the console application class, initialize the assembly and class, and add the appropriate declarations to the CodeDOM graph.

```
C#

public Sample()
{
    targetUnit = new CodeCompileUnit();
    CodeNamespace samples = new CodeNamespace("CodeDOMSample");
    samples.Imports.Add(new CodeNamespaceImport("System"));
    targetClass = new CodeTypeDeclaration("CodeDOMCreatedClass");
    targetClass.IsClass = true;
    targetClass.TypeAttributes =
        TypeAttributes.Public | TypeAttributes.Sealed;
    samples.Types.Add(targetClass);
    targetUnit.Namespaces.Add(samples);
}
```

Add members to the CodeDOM graph

- Add fields to the CodeDOM graph by adding [CodeMemberField](#) objects to the [Members](#) property of the class.

```
C#

public void AddFields()
{
    // Declare the widthValue field.
    CodeMemberField widthValueField = new CodeMemberField();
    widthValueField.Attributes = MemberAttributes.Private;
    widthValueField.Name = "widthValue";
}
```

```

        widthValueField.Type = new
CodeTypeReference(typeof(System.Double));
        widthValueField.Comments.Add(new CodeCommentStatement(
            "The width of the object."));
        targetClass.Members.Add(widthValueField);

        // Declare the heightValue field
        CodeMemberField heightValueField = new CodeMemberField();
        heightValueField.Attributes = MemberAttributes.Private;
        heightValueField.Name = "heightValue";
        heightValueField.Type =
            new CodeTypeReference(typeof(System.Double));
        heightValueField.Comments.Add(new CodeCommentStatement(
            "The height of the object."));
        targetClass.Members.Add(heightValueField);
    }

```

- Add properties to the CodeDOM graph by adding [CodeMemberProperty](#) objects to the [Members](#) property of the class.

C#

```

public void AddProperties()
{
    // Declare the read-only Width property.
    CodeMemberProperty widthProperty = new CodeMemberProperty();
    widthProperty.Attributes =
        MemberAttributes.Public | MemberAttributes.Final;
    widthProperty.Name = "Width";
    widthProperty.HasGet = true;
    widthProperty.Type = new CodeTypeReference(typeof(System.Double));
    widthProperty.Comments.Add(new CodeCommentStatement(
        "The Width property for the object."));
    widthProperty.GetStatements.Add(new CodeMethodReturnStatement(
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "widthValue")));
    targetClass.Members.Add(widthProperty);

    // Declare the read-only Height property.
    CodeMemberProperty heightProperty = new CodeMemberProperty();
    heightProperty.Attributes =
        MemberAttributes.Public | MemberAttributes.Final;
    heightProperty.Name = "Height";
    heightProperty.HasGet = true;
    heightProperty.Type = new CodeTypeReference(typeof(System.Double));
    heightProperty.Comments.Add(new CodeCommentStatement(
        "The Height property for the object."));
    heightProperty.GetStatements.Add(new CodeMethodReturnStatement(
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "heightValue")));
    targetClass.Members.Add(heightProperty);

    // Declare the read only Area property.

```

```

CodeMemberProperty areaProperty = new CodeMemberProperty();
areaProperty.Attributes =
    MemberAttributes.Public | MemberAttributes.Final;
areaProperty.Name = "Area";
areaProperty.HasGet = true;
areaProperty.Type = new CodeTypeReference(typeof(System.Double));
areaProperty.Comments.Add(new CodeCommentStatement(
    "The Area property for the object."));

// Create an expression to calculate the area for the get accessor
// of the Area property.
CodeBinaryOperatorExpression areaExpression =
    new CodeBinaryOperatorExpression(
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "widthValue"),
        CodeBinaryOperatorType.Multiply,
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "heightValue"));
areaProperty.GetStatements.Add(
    new CodeMethodReturnStatement(areaExpression));
targetClass.Members.Add(areaProperty);
}

```

- Add a method to the CodeDOM graph by adding a [CodeMemberMethod](#) object to the [Members](#) property of the class.

```

C#

public void AddMethod()
{
    // Declaring a ToString method
    CodeMemberMethod toStringMethod = new CodeMemberMethod();
    toStringMethod.Attributes =
        MemberAttributes.Public | MemberAttributes.Override;
    toStringMethod.Name = "ToString";
    toStringMethod.ReturnType =
        new CodeTypeReference(typeof(System.String));

    CodeFieldReferenceExpression widthReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Width");
    CodeFieldReferenceExpression heightReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Height");
    CodeFieldReferenceExpression areaReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Area");

    // Declaring a return statement for method ToString.
    CodeMethodReturnStatement returnStatement =
        new CodeMethodReturnStatement();

    // This statement returns a string representation of the width,

```



```

// height, and area.
string formattedOutput = "The object:" + Environment.NewLine +
    " width = {0}," + Environment.NewLine +
    " height = {1}," + Environment.NewLine +
    " area = {2}";
returnStatement.Expression =
    new CodeMethodInvokeExpression(
        new CodeTypeReferenceExpression("System.String"), "Format",
        new CodePrimitiveExpression(formattedOutput),
        widthReference, heightReference, areaReference);
toStringMethod.Statements.Add(returnStatement);
targetClass.Members.Add(toStringMethod);
}

```

- Add a constructor to the CodeDOM graph by adding a [CodeConstructor](#) object to the [Members](#) property of the class.

```

C#

public void AddConstructor()
{
    // Declare the constructor
    CodeConstructor constructor = new CodeConstructor();
    constructor.Attributes =
        MemberAttributes.Public | MemberAttributes.Final;

    // Add parameters.
    constructor.Parameters.Add(new CodeParameterDeclarationExpression(
        typeof(System.Double), "width"));
    constructor.Parameters.Add(new CodeParameterDeclarationExpression(
        typeof(System.Double), "height"));

    // Add field initialization logic
    CodeFieldReferenceExpression widthReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "widthValue");
    constructor.Statements.Add(new CodeAssignStatement(widthReference,
        new CodeArgumentReferenceExpression("width")));
    CodeFieldReferenceExpression heightReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "heightValue");
    constructor.Statements.Add(new CodeAssignStatement(heightReference,
        new CodeArgumentReferenceExpression("height")));
    targetClass.Members.Add(constructor);
}

```

- Add an entry point to the CodeDOM graph by adding a [CodeEntryPointMethod](#) object to the [Members](#) property of the class.

```

C#

```

```

public void AddEntryPoint()
{
    CodeEntryPointMethod start = new CodeEntryPointMethod();
    CodeObjectCreateExpression objectCreate =
        new CodeObjectCreateExpression(
            new CodeTypeReference("CodeDOMCreatedClass"),
            new CodePrimitiveExpression(5.3),
            new CodePrimitiveExpression(6.9));

    // Add the statement:
    // "CodeDOMCreatedClass testClass =
    //     new CodeDOMCreatedClass(5.3, 6.9);"
    start.Statements.Add(new CodeVariableDeclarationStatement(
        new CodeTypeReference("CodeDOMCreatedClass"), "testClass",
        objectCreate));

    // Creat the expression:
    // "testClass.ToString()"
    CodeMethodInvokeExpression toStringInvoke =
        new CodeMethodInvokeExpression(
            new CodeVariableReferenceExpression("testClass"), "ToString");

    // Add a System.Console.WriteLine statement with the previous
    // expression as a parameter.
    start.Statements.Add(new CodeMethodInvokeExpression(
        new CodeTypeReferenceExpression("System.Console"),
        "WriteLine", toStringInvoke));
    targetClass.Members.Add(start);
}

```

Generate the code from the CodeDOM graph

Generate source code from the CodeDOM graph by calling the [GenerateCodeFromCompileUnit](#) method.

C#

```

public void GenerateCSharpCode(string fileName)
{
    CodeDomProvider provider = CodeDomProvider.CreateProvider("CSharp");
    CodeGeneratorOptions options = new CodeGeneratorOptions();
    options.BracingStyle = "C";
    using (StreamWriter sourceWriter = new StreamWriter(fileName))
    {
        provider.GenerateCodeFromCompileUnit(
            targetUnit, sourceWriter, options);
    }
}

```

Create the graph and generate the code

1. Add the methods created in the preceding steps to the `Main` method defined in the first step.

```
C#

static void Main()
{
    Sample sample = new Sample();
    sample.AddFields();
    sample.AddProperties();
    sample.AddMethod();
    sample.AddConstructor();
    sample.AddEntryPoint();
    sample.GenerateCSharpCode(outputFileName);
}
```

2. Compile and execute the generating class.

Example

The following code example shows the code from the preceding steps.

```
C#

using System;
using System.Reflection;
using System.IO;
using System.CodeDom;
using System.CodeDom.Compiler;
using Microsoft.CSharp;

namespace SampleCodeDom
{
    /// <summary>
    /// This code example creates a graph using a CodeCompileUnit and
    /// generates source code for the graph using the CSharpCodeProvider.
    /// </summary>
    class Sample
    {
        /// <summary>
        /// Define the compile unit to use for code generation.
        /// </summary>
        CodeCompileUnit targetUnit;

        /// <summary>
        /// The only class in the compile unit. This class contains 2
        fields,
        /// 3 properties, a constructor, an entry point, and 1 simple
```

```

method.
    /// </summary>
    CodeTypeDeclaration targetClass;

    /// <summary>
    /// The name of the file to contain the source code.
    /// </summary>
    private const string outputFileName = "SampleCode.cs";

    /// <summary>
    /// Define the class.
    /// </summary>
    public Sample()
    {
        targetUnit = new CodeCompileUnit();
        CodeNamespace samples = new CodeNamespace("CodeDOMSample");
        samples.Imports.Add(new CodeNamespaceImport("System"));
        targetClass = new CodeTypeDeclaration("CodeDOMCreatedClass");
        targetClass.IsClass = true;
        targetClass.TypeAttributes =
            TypeAttributes.Public | TypeAttributes.Sealed;
        samples.Types.Add(targetClass);
        targetUnit.Namespaces.Add(samples);
    }

    /// <summary>
    /// Adds two fields to the class.
    /// </summary>
    public void AddFields()
    {
        // Declare the widthValue field.
        CodeMemberField widthValueField = new CodeMemberField();
        widthValueField.Attributes = MemberAttributes.Private;
        widthValueField.Name = "widthValue";
        widthValueField.Type = new
CodeTypeReference(typeof(System.Double));
        widthValueField.Comments.Add(new CodeCommentStatement(
            "The width of the object."));
        targetClass.Members.Add(widthValueField);

        // Declare the heightValue field
        CodeMemberField heightValueField = new CodeMemberField();
        heightValueField.Attributes = MemberAttributes.Private;
        heightValueField.Name = "heightValue";
        heightValueField.Type =
            new CodeTypeReference(typeof(System.Double));
        heightValueField.Comments.Add(new CodeCommentStatement(
            "The height of the object."));
        targetClass.Members.Add(heightValueField);
    }

    /// <summary>
    /// Add three properties to the class.
    /// </summary>
    public void AddProperties()
    {

```

```

// Declare the read-only Width property.
CodeMemberProperty widthProperty = new CodeMemberProperty();
widthProperty.Attributes =
    MemberAttributes.Public | MemberAttributes.Final;
widthProperty.Name = "Width";
widthProperty.HasGet = true;
widthProperty.Type = new
CodeTypeReference(typeof(System.Double));
widthProperty.Comments.Add(new CodeCommentStatement(
    "The Width property for the object."));
widthProperty.GetStatements.Add(new CodeMethodReturnStatement(
    new CodeFieldReferenceExpression(
        new CodeThisReferenceExpression(), "widthValue")));
targetClass.Members.Add(widthProperty);

// Declare the read-only Height property.
CodeMemberProperty heightProperty = new CodeMemberProperty();
heightProperty.Attributes =
    MemberAttributes.Public | MemberAttributes.Final;
heightProperty.Name = "Height";
heightProperty.HasGet = true;
heightProperty.Type = new
CodeTypeReference(typeof(System.Double));
heightProperty.Comments.Add(new CodeCommentStatement(
    "The Height property for the object."));
heightProperty.GetStatements.Add(new CodeMethodReturnStatement(
    new CodeFieldReferenceExpression(
        new CodeThisReferenceExpression(), "heightValue")));
targetClass.Members.Add(heightProperty);

// Declare the read only Area property.
CodeMemberProperty areaProperty = new CodeMemberProperty();
areaProperty.Attributes =
    MemberAttributes.Public | MemberAttributes.Final;
areaProperty.Name = "Area";
areaProperty.HasGet = true;
areaProperty.Type = new
CodeTypeReference(typeof(System.Double));
areaProperty.Comments.Add(new CodeCommentStatement(
    "The Area property for the object."));

// Create an expression to calculate the area for the get
accessor
// of the Area property.
CodeBinaryOperatorExpression areaExpression =
    new CodeBinaryOperatorExpression(
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "widthValue"),
        CodeBinaryOperatorType.Multiply,
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "heightValue"));
areaProperty.GetStatements.Add(
    new CodeMethodReturnStatement(areaExpression));
targetClass.Members.Add(areaProperty);
}

```

```

/// <summary>
/// Adds a method to the class. This method multiplies values stored
/// in both fields.
/// </summary>
public void AddMethod()
{
    // Declaring a ToString method
    CodeMemberMethod toStringMethod = new CodeMemberMethod();
    toStringMethod.Attributes =
        MemberAttributes.Public | MemberAttributes.Override;
    toStringMethod.Name = "ToString";
    toStringMethod.ReturnType =
        new CodeTypeReference(typeof(System.String));

    CodeFieldReferenceExpression widthReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Width");
    CodeFieldReferenceExpression heightReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Height");
    CodeFieldReferenceExpression areaReference =
        new CodeFieldReferenceExpression(
            new CodeThisReferenceExpression(), "Area");

    // Declaring a return statement for method ToString.
    CodeMethodReturnStatement returnStatement =
        new CodeMethodReturnStatement();

    // This statement returns a string representation of the width,
    // height, and area.
    string formattedOutput = "The object:" + Environment.NewLine +
        " width = {0}," + Environment.NewLine +
        " height = {1}," + Environment.NewLine +
        " area = {2}";
    returnStatement.Expression =
        new CodeMethodInvokeExpression(
            new CodeTypeReferenceExpression("System.String"), "Format",
            new CodePrimitiveExpression(formattedOutput),
            widthReference, heightReference, areaReference);
    toStringMethod.Statements.Add(returnStatement);
    targetClass.Members.Add(toStringMethod);
}

/// <summary>
/// Add a constructor to the class.
/// </summary>
public void AddConstructor()
{
    // Declare the constructor
    CodeConstructor constructor = new CodeConstructor();
    constructor.Attributes =
        MemberAttributes.Public | MemberAttributes.Final;

    // Add parameters.
    constructor.Parameters.Add(new

```

```

CodeParameterDeclarationExpression(
    typeof(System.Double), "width"));
    constructor.Parameters.Add(new
CodeParameterDeclarationExpression(
    typeof(System.Double), "height"));

    // Add field initialization logic
CodeFieldReferenceExpression widthReference =
    new CodeFieldReferenceExpression(
        new CodeThisReferenceExpression(), "widthValue");
    constructor.Statements.Add(new
CodeAssignStatement(widthReference,
    new CodeArgumentReferenceExpression("width")));
CodeFieldReferenceExpression heightReference =
    new CodeFieldReferenceExpression(
        new CodeThisReferenceExpression(), "heightValue");
    constructor.Statements.Add(new
CodeAssignStatement(heightReference,
    new CodeArgumentReferenceExpression("height")));
    targetClass.Members.Add(constructor);
}

/// <summary>
/// Add an entry point to the class.
/// </summary>
public void AddEntryPoint()
{
    CodeEntryPointMethod start = new CodeEntryPointMethod();
    CodeObjectCreateExpression objectCreate =
        new CodeObjectCreateExpression(
            new CodeTypeReference("CodeDOMCreatedClass"),
            new CodePrimitiveExpression(5.3),
            new CodePrimitiveExpression(6.9));

    // Add the statement:
    // "CodeDOMCreatedClass testClass =
    //     new CodeDOMCreatedClass(5.3, 6.9);"
    start.Statements.Add(new CodeVariableDeclarationStatement(
        new CodeTypeReference("CodeDOMCreatedClass"), "testClass",
        objectCreate));

    // Creat the expression:
    // "testClass.ToString()"
    CodeMethodInvokeExpression toStringInvoke =
        new CodeMethodInvokeExpression(
            new CodeVariableReferenceExpression("testClass"),
            "ToString");

    // Add a System.Console.WriteLine statement with the previous
    // expression as a parameter.
    start.Statements.Add(new CodeMethodInvokeExpression(
        new CodeTypeReferenceExpression("System.Console"),
        "WriteLine", toStringInvoke));
    targetClass.Members.Add(start);
}

```

```

    /// <summary>
    /// Generate CSharp source code from the compile unit.
    /// </summary>
    /// <param name="filename">Output file name</param>
    public void GenerateCSharpCode(string fileName)
    {
        CodeDomProvider provider =
CodeDomProvider.CreateProvider("CSharp");
        CodeGeneratorOptions options = new CodeGeneratorOptions();
        options.BracingStyle = "C";
        using (StreamWriter sourceWriter = new StreamWriter(fileName))
        {
            provider.GenerateCodeFromCompileUnit(
                targetUnit, sourceWriter, options);
        }
    }

    /// <summary>
    /// Create the CodeDOM graph and generate the code.
    /// </summary>
    static void Main()
    {
        Sample sample = new Sample();
        sample.AddFields();
        sample.AddProperties();
        sample.AddMethod();
        sample.AddConstructor();
        sample.AddEntryPoint();
        sample.GenerateCSharpCode(outputFileName);
    }
}

```

When the preceding example is compiled and executed, it produces the following source code.

C#

```

//-----
// <auto-generated>
//   This code was generated by a tool.
//   Runtime Version:2.0.50727.42
//
//   Changes to this file may cause incorrect behavior and will be lost if
//   the code is regenerated.
// </auto-generated>
//-----

namespace CodeDOMSample
{
    using System;

```



```

public sealed class CodeDOMCreatedClass
{

    // The width of the object.
    private double widthValue;

    // The height of the object.
    private double heightValue;

    public CodeDOMCreatedClass(double width, double height)
    {
        this.widthValue = width;
        this.heightValue = height;
    }

    // The Width property for the object.
    public double Width
    {
        get
        {
            return this.widthValue;
        }
    }

    // The Height property for the object.
    public double Height
    {
        get
        {
            return this.heightValue;
        }
    }

    // The Area property for the object.
    public double Area
    {
        get
        {
            return (this.widthValue * this.heightValue);
        }
    }

    public override string ToString()
    {
        return string.Format(
            "The object:\r\n width = {0},\r\n height = {1},\r\n area = {2}",
            this.Width, this.Height, this.Area);
    }

    public static void Main()
    {
        CodeDOMCreatedClass testClass = new CodeDOMCreatedClass(5.3,
6.9);
        System.Console.WriteLine(testClass.ToString());
    }
}

```

```
}  
    }  
}
```

The generated source code produces the following output when compiled and executed.

Output

```
The object:  
width = 5.3,  
height = 6.9,  
area = 36.57
```

Compiling the Code

- This code example requires the `FullTrust` permission set to execute successfully.

See also

- [Using the CodeDOM](#)
- [Generating and Compiling Source Code from a CodeDOM Graph](#)